

学习（在测试时学习）：具有 表达性隐藏状态的循环神经网络

Yu Sun^{*1}, Xinhao Li^{*2}, Karan Dalal^{*3},
Jiarui Xu², Arjun Vikram¹, Genghan Zhang¹, Yann Dubois¹,
Xinlei Chen^{†4}, Xiaolong Wang^{†2}, Sanmi Koyejo^{†1}, Tatsunori Hashimoto^{†1}, Carlos Guestrin^{†1}
¹ Stanford University ² UC San Diego ³ UC Berkeley ⁴ Meta AI

摘要

自注意力（Self-attention）在长上下文中表现良好，但具有二次复杂度。现有的循环神经网络（RNN）层具有线性复杂度，但其在长上下文中的性能受限于隐藏状态的表达能力。本文提出一个实用的框架，用于实例化具有线性复杂度和强表达力隐藏状态的序列建模层。其核心思想是将隐藏状态本身设计为一个机器学习模型，并将更新规则设计为自监督学习的一个步骤。由于隐藏状态即使在测试序列上也会通过训练进行更新，因此本文所提出的层被称为测试时训练（Test-Time Training）层。

（测试时训练）层。本文考虑两种实例化：测试时训练线性模型和测试时训练多层感知机，其隐藏状态分别为线性模型和两层多层感知机。本文在 1.25 亿至 13 亿参数规模下评估这些实例化，并与强基线变换器（Transformer）及现代循环神经网络曼巴（Mamba）进行对比。与变换器类似，测试时训练线性模型和测试时训练多层感知机能够通过基于更多词元进行条件化来持续降低困惑度，而曼巴在 16k 上下文后无法做到。测试时训练多层感知机在内存输入输出方面仍面临挑战，但在长上下文中展现出更大潜力，为未来研究指明了一个有前景的方向。

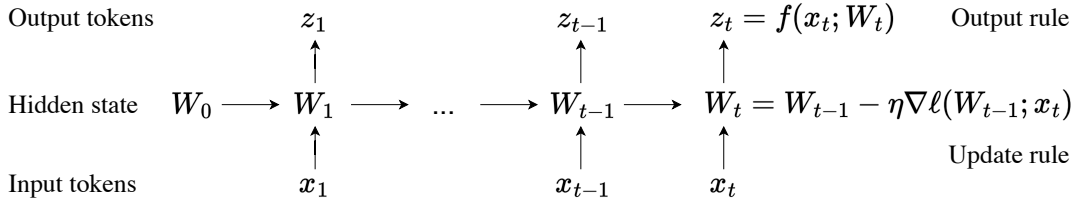


图 1。所有序列建模层均可表示为一个根据更新规则进行状态转移的隐藏状态。本文的核心思想是将隐藏状态本身视为一个具有权重 W 的模型 f ，并将更新规则视为自监督损失上的梯度步 ℓ 。因此，在测试序列上更新隐藏状态等价于在测试时训练模型 f 。这一过程被称为测试时训练（Test-Time Training, TTT），并被编程到本文的 TTT 层中。

*Core 贡献者。† 联合指导。作者贡献详见文末。通讯作者：ys646@stanford.edu, xil202@ucsd.edu, kdalal@berkeley.edu。代码提供 JAX 和 PyTorch 版本。本文初版于 2024 年 7 月 5 日提交至 arXiv。当前版本更新了相关工作与局限性。所有实验均完成于初版。

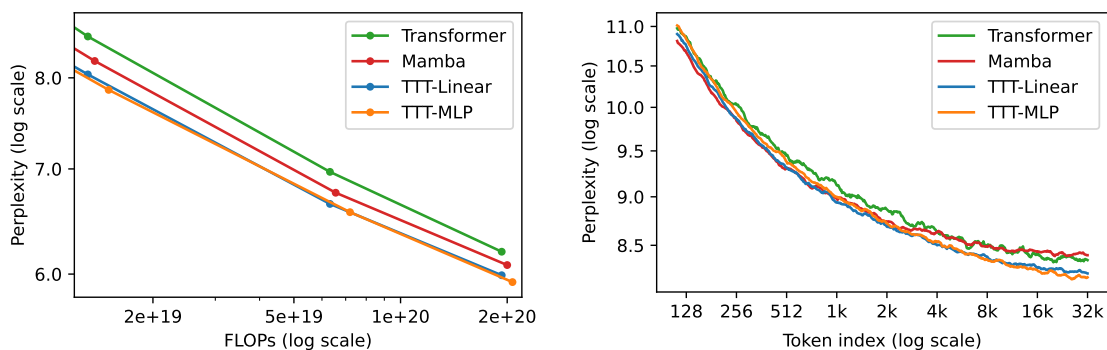


图 2. 与曼巴 (Mamba) 相比, TTT-线性 (TTT-Linear) 和 TTT-多层感知机 (TTT-MLP) 在 8k 上下文 (左) 中具有相似的困惑度, 并且能更好地利用长上下文 (右)。评估遵循卡普兰等人 (Kaplan et al.) [43]。左: 在 8k 上下文的堆 (Pile) 上的缩放趋势, 放大到 3.5 亿至 13 亿参数之间。右: 与变换器 (Transformer) 类似, TTT-线性 和 TTT-多层感知机可以通过基于更多标记进行条件化来持续降低困惑度, 而曼巴在 16k 上下文之后则无法做到。所有方法都匹配了曼巴 14 亿参数的训练浮点运算次数 (FLOPs)。

1 引言

2020 年, 开放人工智能 (OpenAI) 的缩放定律论文 (卡普兰等人 [43]) 表明, 长短期记忆网络 (LSTM, 一种循环神经网络) 无法像变换器那样扩展, 也无法有效利用长上下文。现在, 借助现代循环神经网络和最佳实践, 我们在图 2 中重新评估这些发现。

在左侧, 我们观察到曼巴 [27]——当今最流行的循环神经网络之一——其扩展方式与强大的变换器相似, 显示出自 2020 年长短期记忆网络以来的巨大进步。然而, 在右侧, 我们观察到曼巴存在与卡普兰等人在长短期记忆网络中发现的相同问题。序列中靠后的标记平均而言应该更容易预测, 因为它们基于更多信息进行条件化。变换器确实如此, 其每个标记索引处的平均困惑度在整个 32k 上下文中持续下降。相比之下, 曼巴的同一度量在 16k 之后趋于平稳。

这一结果代表了现有循环神经网络的一个尴尬现实。一方面, 循环神经网络 (相对于变换器) 的主要优势是其线性 (相对于二次) 复杂度。这种渐近优势只有在长上下文的实践中才能实现, 根据图 12, 这发生在 8k 之后。另一方面, 一旦上下文足够长, 现有的循环神经网络 (如曼巴) 却难以真正利用所条件化的额外信息。

长上下文的困难是循环神经网络层本质所固有的: 与自注意力不同, 循环神经网络层必须将上下文压缩到固定大小的隐藏状态中。作为一种压缩启发式方法, 更新规则需要发现数千甚至数百万个标记之间的底层结构和关系。这种需求本质上具有挑战性。在本文中, 我们从一个观察开始: 自监督学习可以将大规模训练集压缩到模型 (如大语言模型) 的权重中, 该模型通常对其训练数据之间的语义联系表现出深刻的理解——这正是我们需要的压缩启发式方法。

测试时训练 (TTT) 层。受此观察启发, 我们将隐藏状态本身设计为一个机器学习模型, 并将更新规则设计为自监督学习的一个步骤。由于隐藏状态即使在测试序列上也会通过训练进行更新, 因此这些循环神经网络层被称为测试时训练 (Test-Time Training, TTT) 层。我们引入两种简单的实例化: TTT-线性层和 TTT-多层感知机层, 其中隐藏状态分别是一个线性模型和一个两层多层感知机。TTT 层可以集成到任何网络架构中, 并像循环神经网络层和自注意力一样进行端到端优化。

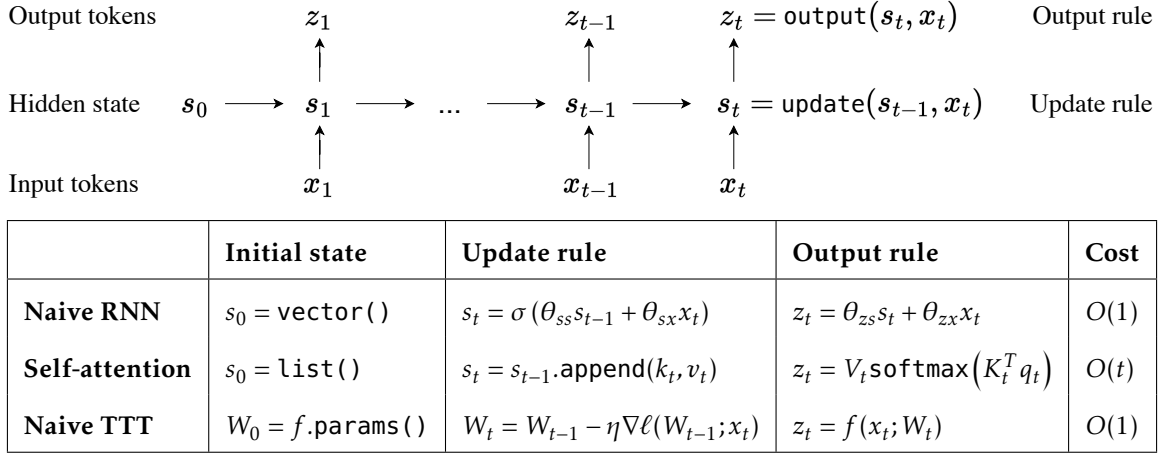


图 3. 顶部：一个通用的序列建模层，表示为根据更新规则进行状态转移的隐藏状态。所有序列建模层都可以看作图中三个组成部分的不同实例化：初始状态、更新规则和输出规则。底部：序列建模层的示例及其三个组成部分的实例化。朴素的 TTT 层如图 1 所示。自注意力的隐藏状态随上下文增长，因此每个标记的成本也随之增长。朴素的循环神经网络和 TTT 层都将不断增长的上下文压缩为固定大小的隐藏状态，因此每个标记的成本保持不变。

实际运行时间。我们采用两种技术使 TTT 层在现代图形处理器和张量处理器上更加高效。首先，类似于在常规训练中对序列小批量进行梯度步骤以获得更好并行性的标准做法，我们在 TTT 期间使用标记的小批量。其次，我们为每个 TTT 小批量内的操作开发了一种对偶形式。该对偶形式在输出上与朴素实现等价，但在我们的张量处理器上训练速度提高了 $5 \times$ 倍以上。

贡献与局限性。使用线性模型作为隐藏状态的思想已经在 DeltaNet[62, 83] 中得到了充分研究。自我们的第一版发布以来，具有矩阵（线性）隐藏状态的循环神经网络层在 Mamba 2[17] 和门控 DeltaNet[82] 中也得到了进一步发展。与这一系列工作相比，我们的贡献在于提供了一个实用的框架，可以将任意神经网络实例化为隐藏状态。然而，即使在我们提高了效率之后，此类实例化仍可能需要大量的实际运行时间。我们的框架能否产生克服这一局限性的实例化，或提供超过其代价的收益，仍有待观察。

2 方法

所有序列建模层都可以从将历史上下文存储到隐藏状态的角度来看待，如图 3.¹ 所示。例如，循环神经网络（RNN）层——如长短期记忆网络（LSTM）[33]、RWKV [59] 和 Mamba [27] 层——将上下文压缩为跨时间固定大小的状态。这种压缩带来两个后果。一方面，将输入标记 x_t 映射到输出标记 z_t 是高效的，因为更新规则和输出规则对每个标记都只需常数时间。另一方面，循环神经网络层在长上下文中的性能受限于其隐藏状态 s_t 的表达能力。

自注意力也可以从上述角度来理解，只不过其隐藏状态通常称为键值（KV）缓存，是一个随 t 线性增长的列表。其更新规则只是将当前的键值元组追加到这个列表中，而输出规则扫描所有直到 t 的元组以形成

¹ 我们将序列建模层定义为从一个序列到另一个序列的自回归映射。

注意力矩阵。隐藏状态显式地存储所有历史上下文而不进行压缩，这使得自注意力在长上下文中比循环神经网络层更具表达能力。然而，扫描这个线性增长的隐藏状态也需要每个标记线性增长的时间。

为了在长上下文中同时保持高效和表达力，我们需要一种更好的压缩启发式方法。具体来说，我们需要将数千甚至数百万个标记压缩到一个能够有效捕捉其底层结构和关系的隐藏状态中。这听起来可能要求很高，但实际上我们所有人都已经熟悉这样一种启发式方法。

2.1 测试时训练作为隐藏状态更新

参数化学习的过程可以看作是将大规模训练集压缩到模型权重中。具体来说，我们知道通过自监督训练的模型能够捕捉其训练数据背后的底层结构和关系 [51]——这正是我们需要的压缩启发式方法。

大语言模型（LLM）本身就是很好的例子。通过下一标记预测的自监督任务进行训练，它们的权重可以看作是对互联网上现有知识的压缩存储形式。通过查询大语言模型，我们可以从它们的权重中提取知识。更重要的是，大语言模型通常表现出对现有知识之间语义联系的深刻理解，从而表达出新的推理片段 [1]。

我们的核心思想是利用自监督学习将历史上下文 $x_1, \dots, x_t$ 压缩为隐藏状态 s_t ，通过将上下文视为无标签数据集、将状态视为模型来实现。具体而言，隐藏状态 s_t 现在等价于 W_t ，即模型 f 的权重，该模型可以是线性模型、小型神经网络或其他任何形式。输出规则简单表示为：

$$z_t = f(x_t; W_t). \quad (1)$$

直观上，输出标记就是 f 使用更新后的权重 W_t 对 x_t 所做的预测。更新规则是在某个自监督损失 ℓ 上执行一步梯度下降：

$$W_t = W_{t-1} - \eta \nabla \ell(W_{t-1}; x_t), \quad (2)$$

学习率为 η 。² 从压缩的角度看，每种启发式方法都需要决定记住或遗忘哪些输入。我们的 W 会记住产生较大梯度的输入——直观地说，就是那些让 W 学到很多内容的输入。

ℓ 的一种选择是重建 x_t 本身。为了使学习问题具有非平凡性，我们首先将 x_t 处理为损坏的输入 $\tilde{x}_t$ （细节见第 2.3 小节），然后优化：

$$\ell(W; x_t) = \|f(\tilde{x}_t; W) - x_t\|^2. \quad (3)$$

与去噪自编码器类似 [77]， f 需要发现 x_t 各维度之间的相关性，才能从部分信息 $\tilde{x}_t$ 中重建它。³ 如图 4 所示，梯度下降能够降低 ℓ ，但无法将其降至零。我们在第 2.3 小节讨论更复杂的自监督任务形式。

与其他循环神经网络层和自注意力机制一样，我们将输入序列 $x_1, \dots, x_T$ 映射到输出序列 $z_1, \dots, z_T$ 的算法可以编程到序列建模层的前向传播中，使用上述隐藏状态、更新规则和输出规则。即使在测试时，我们的新层仍会为每个输入序列训练不同的权重序列 $W_1, \dots, W_T$ 。因此，我们称之为测试时训练（TTT）层。

² 目前，考虑 $W_0 = 0$ 。我们将在第 2.7 小节讨论更复杂的 W 初始化技术。³ 在过去的实验中，我们也尝试过在 f （编码器）之后添加另一个模型 g （解码器），使得重建由 $g \circ f$ 产生，而不仅仅由 f 本身产生。虽然这种更重的设计略微改善了结果，但它使整体训练稳定性下降，并显著增加了计算成本。因此，我们专注于仅编码器的设计。

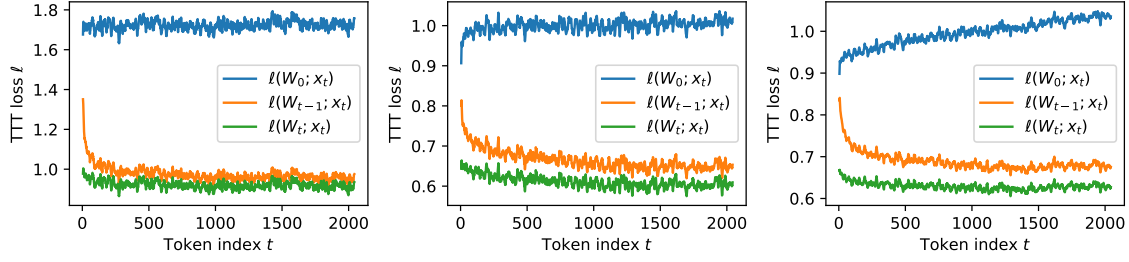


图 4. 自监督 TTT 损失 ℓ 在形式为 $x_1, \dots, x_T$ 的所有测试序列上的平均值，其中 $T = 2048$ ，针对具有 1.25 亿参数网络的前三个 TTT 层。一步梯度下降能够将 TTT 损失从 $\ell(W_{t-1}; x_t)$ 降至 $\ell(W_t; x_t)$ 。随着 t 沿测试序列进一步移动， $\ell(W_t; x_t)$ 也从 $\ell(W_0; x_t)$ 进一步改善。为清晰起见，损失值已在 10 个时间步的滑动窗口上取平均。所有 12 层的完整结果见图 14（见附录）。

2.2 训练具有 TTT 层的网络 TTT 层的前向传播也有对应的反向传播。我们的前向传播仅由标准可微算子组成，除了梯度算子 ∇ 。然而， ∇_{just} 将一个函数映射到另一个函数，在这种情况下 ℓ 到 $\nabla \ell$ ，并且 $\nabla \ell$ 也由可微算子组成。从概念上讲，对 $\nabla \ell$ 调用反向传播意味着对梯度求梯度——这是元学习 [54] 中一项已被充分研究的技术。

TTT 层具有与 RNN 层和自注意力相同的接口，因此可以在任何更大的网络架构中替换，这些架构通常包含许多这样的序列建模层。训练具有 TTT 层的网络也与训练任何其他语言模型（如变换器）的方式相同。可以使用相同的数据、配方和目标（如下一个词元预测）来优化网络其余部分的参数。

我们将训练更大的网络称为外循环，将每个 TTT 层内训练 W 称为内循环。这两个嵌套学习问题之间的一个重要区别是，内循环梯度 $\nabla \ell$ 是关于 W （ f 的参数）取的，而外循环梯度是关于网络其余部分的参数取的，我们将其记为 θ_{rest} 。在本文中，外循环参数始终用带有各种下标的 θ 表示。

到目前为止，与其他 RNN 层和自注意力相比，TTT 层没有外循环参数。在第 2.3 小节中，我们为 TTT 层添加外循环参数以改进其自监督任务。然后在第 2.4 和 2.5 小节中，我们讨论两种改善 TTT 层实际运行时间的方法。

2.3 为测试时训练学习自监督任务 可以说，测试时训练最重要的部分是自监督任务，因为它决定了 W 将从测试序列中学习到的特征类型。那么我们应该如何设计这个任务呢？测试时训练的最终目标是使 $z_t = f(x_t; W_t)$ 在语言建模上表现良好。我们没有根据人类先验手工设计自监督任务，而是采用更端到端的方法——直接针对下一个词元预测的最终目标优化自监督任务。

具体来说，我们将自监督任务作为外循环的一部分来学习。从方程 3 中的朴素重建任务出发，我们添加一些外循环参数使该任务可学习。在 2.1 小节中，我们没有指定从 x_t 产生 $\tilde{x}_t$ 的损坏方式。一种设计是将其设为低秩投影 $\tilde{x}_t = \theta_K x_t$ ，其中 θ_K 是一个可学习矩阵。⁴ 按照多视图重建的术语， $\theta_K x_t$ 被称为训练视图 [13]。

⁴ 下标 K 暗示了与自注意力的联系，我们将在 2.6 小节中建立这一联系。

```

class TTT_Layer(nn.Module):
    def __init__(self):
        self.task = Task()

    def forward(self, in_seq):
        state = Learner(self.task)
        out_seq = []
        for tok in in_seq:
            state.train(tok)
            out_seq.append(state.predict(tok))
        return out_seq

class Task(nn.Module):
    def __init__(self):
        self.theta_K = nn.Param((d1, d2))
        self.theta_V = nn.Param((d1, d2))
        self.theta_Q = nn.Param((d1, d2))

    def loss(self, f, x):
        train_view = self.theta_K @ x
        label_view = self.theta_V @ x
        return MSE(f(train_view), label_view)

class Learner():
    def __init__(self, task):
        self.task = task
        # Linear here, but can be any model
        self.model = Linear()
        # online GD here for simplicity
        self.optim = OGD()

    def train(self, x):
        # grad function wrt first arg
        # of loss, which is self.model
        grad_fn = grad(self.task.loss)
        # calculate inner-loop grad
        grad_in = grad_fn(self.model, x)

        # starting from current params,
        # step in direction of grad_in,
        self.optim.step(self.model, grad_in)

    def predict(self, x):
        test_view = self.task.theta_Q @ x
        return self.model(test_view)

```

图 5. 以 PyTorch 风格实现的带有线性模型和在线梯度下降的测试时训练层的朴素实现。测试时训练 _ 层可以像其他序列建模层一样嵌入更大的网络中。训练网络将优化测试时训练 _ 层中 Task 的参数，因为两者都是 nn.Module 的子类。由于 Learner 不是 nn.Module 的子类，state.model 在内循环中每次调用 state.train 时手动更新。为简单起见，我们有时将 model 重载为 model.parameters。

此外，也许并非 x_t 中的所有信息都值得记住，因此重建标签可以是另一个低秩投影 $\theta_V x_t$ ，而不是 x_t 。这里 $\theta_V x_t$ 被称为标签视图，其中 θ_V 也是可学习的。总之，我们新的自监督损失为：

$$\ell(W; x_t) = \|f(\theta_K x_t; W) - \theta_V x_t\|^2. \quad (4)$$

由于 W 和各种 θ 同时出现在方程 4 中，我们再次强调它们在本质上的区别。在内循环中，只有 W 被优化，因此写作 ℓ 的参数； θ_s 是该损失函数的“超参数”。在外循环中， θ_K 、 θ_V 、 θ_Q 与 θ_{rest} 一起被优化，而 W 只是一个隐藏状态，不是参数。图 5 用代码说明了这一区别，其中 θ_K 和 θ_V 被实现为测试时训练层的参数，类似于自注意力的 Key 和 Value 参数。

最后，训练视图 $\theta_K x_t$ 的维度少于 x_t ，因此我们不能再使用方程 1 中的输出规则。最简单的解决方案是创建一个测试视图 $\theta_Q x_t$ ，并将我们的输出规则改为：

$$z_t = f(\theta_Q x_t; W_t). \quad (5)$$

该解决方案还有一个额外的好处。训练视图和标签视图指定了 x_t 中被压缩到 W_t 并随时间向前传播的信息。测试视图指定了可能不同的信息，这些信息被映射到当前输出标记 z_t 并通过网络层向前传播，因此为自监督任务增加了更多的灵活性。

总之，所有可能的 θ_K 、 θ_Q 、 θ_V 选择集合诱导了一个多视图重建任务族，外层循环可以解释为从这个族中选择一个任务。这里为了简单起见，我们将所有视图设计为线性投影。未来的工作可以尝试更灵活的变换，或者更大且不同的自监督任务族。

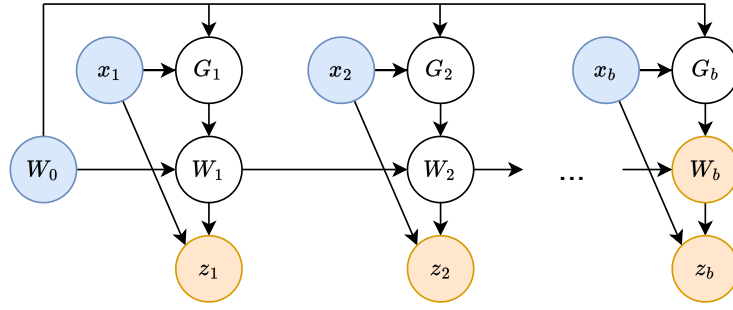


图 6. 第一个 TTT 小批量数据的高层计算图，其中节点是变量，边是计算。蓝色节点是输入变量，黄色是输出变量。第 2.4 小节：由于 $G_1, \dots, G_b$ 之间没有连接，它们彼此之间没有顺序依赖，因此可以并行计算。第 2.5 小节：我们实际上并不物化白色节点——中间的 G 和 W ——来以对偶形式计算输出变量。

2.4 使用小批量 TTT 进行并行化

到目前为止开发的朴素 TTT 层在浮点运算次数（FLOPs）方面已经非常高效。然而，其更新规则 $W_t = W_{t-1} - \eta \nabla l(W_{t-1}; x_t)$ 无法并行化，因为 W_t 在两个地方依赖于 W_{t-1} ：在减号之前和 ∇l 内部。由于 ∇l 包含了大部分计算，我们专注于使这第二部分并行化。

我们通过 TTT 框架中的概念来应对这一系统挑战。梯度下降（GD）有许多变体。GD 的一般更新规则可以表示为：

$$W_t = W_{t-1} - \eta G_t = W_0 - \eta \sum_{s=1}^t G_s, \quad (6)$$

其中 G_t 是下降方向。注意，一旦我们为 $t = 1, \dots, T$ 计算了 G_t ，我们就可以通过方程 6 的后半部分通过累加和获得所有的 W_t 。我们的朴素更新规则，称为在线梯度下降，使用 $G_t = \nabla l(W_{t-1}; x_t)$ 。

为了对 G_t 进行并行化以处理 $t = 1, \dots, T$ ，我们可以对 W_0 取所有梯度。这种带有 $G_t = \nabla l(W_0; x_t)$ 的变体被称为批量梯度下降，因为 $\sum_{s=1}^t \nabla l(W_0; x_s)$ 与对 $x_1, \dots, x_t$ 作为一批的 W_0 的梯度相同。然而，在批量梯度下降中， W_t 实际上只离 W_0 一步梯度，而在线梯度下降中， W_t 离 W_0 有 t 步。因此，批量梯度下降的有效搜索空间更小，这最终损害了语言建模的性能。

我们提出的解决方案——小批量梯度下降——如图 6 所示。将 TTT 批大小记为 b 。我们使用 $G_t = \nabla l(W_{t'}; x_t)$ ，其中 $t' = t - \text{mod}(t, b)$ 是前一个小批量的最后一个时间步（对于第一个小批量则为 0），因此我们可以一次并行计算 b 个梯度。经验上， b 控制速度与质量之间的权衡，如图 7 所示。本文所有实验均选择 $b = 16$ 。

总之，存在两个潜在的信息传播通道，从 W_s 到 W_t ，其中 $s < t$ ：累积和与梯度算子。累积和始终活跃，但梯度通道仅当 W_s 来自前一个小批量时才活跃。梯度下降的不同变体仅影响梯度通道，即下降方向 G_t ，具体是关于哪个 W 取梯度。然而，下降步长 $W_t = W_{t-1} - \eta G_t$ 始终从 W_{t-1} 开始，这是由于更新规则的自回归性质，这与 G_t 的选择无关。

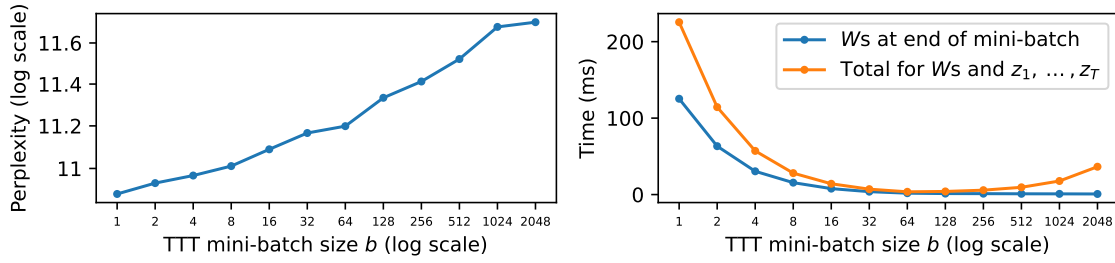


图 7. 对 TTT 小批量大小 b 的消融实验，其中 $b = 1$ 为在线梯度下降， $b = T$ 为批量梯度下降。本文所有实验均选择 $b = 16$ 。左图：较小的 b 改善困惑度，因为执行了更多梯度下降步骤。⁵ $b = 16$ 时的困惑度 11.09 对应于图 10 中 TTT-Linear 的最终结果。右图：对偶形式下的前向时间，上下文长度 $T = 2048$ 。总时间（橙色）可分解为在每个小批量末尾计算 W 的时间（蓝色）和 $z_1, \dots, z_T$ 的时间（橙色 - 蓝色）。⁶ W 的时间复杂度为 $O(T \times d^2)$ ，与 b 无关，但蓝色线随着 b 增大而下降，因为更大的 b 允许更多并行化，直到硬件利用率饱和。 $z_1, \dots, z_T$ 的时间复杂度为 $O(T \times b \times d)$ ，因此橙色线先随着并行化增加而下降，然后随着 $z_1, \dots, z_T$ 的额外计算占主导而上升。

2.5 对偶形式

上述并行化对于实际运行时间的效率而言是必要的，但并非充分条件。现代加速器专长于矩阵-矩阵乘法，即矩阵乘（matmul）。例如，英伟达 A100 图形处理器（GPU）包含高度优化的单元，称为张量核心（TensorCore），它们只能执行单一操作——将两个各为 16×16 大小的矩阵相乘。若缺乏足够的矩阵乘运算，张量核心将处于空闲状态，A100 的大部分潜力便无法发挥。

遗憾的是，迄今为止所开发的 TTT 层，即便采用小批量处理，其矩阵乘运算仍然非常少。考虑最简单的情形 ℓ ，其中 $\theta_K = \theta_V = \theta_Q = I$ ，仅针对首个大小为 b 的 TTT 小批量。此外，将 f 视为线性模型。复制方程 3，在时间 t 处的损失为：

$$\ell(W_0; x_t) = \|f(x_t; W_0) - x_t\|^2 = \|W_0 x_t - x_t\|^2.$$

如第 2.4 小节所讨论的，我们可以并行计算：

$$G_t = \nabla \ell(W_0; x_t) = 2(W_0 x_t - x_t) x_t^T,$$

对于 $t = 1, \dots, b$ 。然而，我们无法通过单次矩阵乘来计算所有 b 个 G_t 。相反，我们需要 b 次外积来逐一计算它们。更糟糕的是，对于每个 $x_t \in \mathbb{R}^d$ ， G_t ，其值为 $d \times d$ ，这比 x_t （当 d 较大时）带来更重的内存占用和输入输出（I/O）成本。

为解决这两个问题，我们提出一个简单的观察：实际上无需显式生成 $G_1, \dots, G_b$ ，只要能在小批量结束时计算出 W_b 以及输出标记 $z_1, \dots, z_b$ （见图 6）。下面以上述简化的 TTT-Linear 情形为例演示这些计算。记 $X = [x_1, \dots, x_b]$ ，则：

$$W_b = W_0 - \eta \sum_{t=1}^b G_t = W_0 - 2\eta \sum_{t=1}^b (W_0 x_t - x_t) x_t^T = W_0 - 2\eta (W_0 X - X) X^T.$$

⁵ 理论上， b 可能过小，导致小批量之间的方差过高，从而损害优化效果。然而，我们在实践中尚未观察到此类效应。⁶ 对于图 7，我们在 TTT-线性 1.3B 中使用单个 TTT 层，并以纯 PyTorch 实现。我们的融合核函数显著提升了时间效率，但使得计算 W_b vs. $z_1, \dots, z_b$ 的时间难以清晰分解。

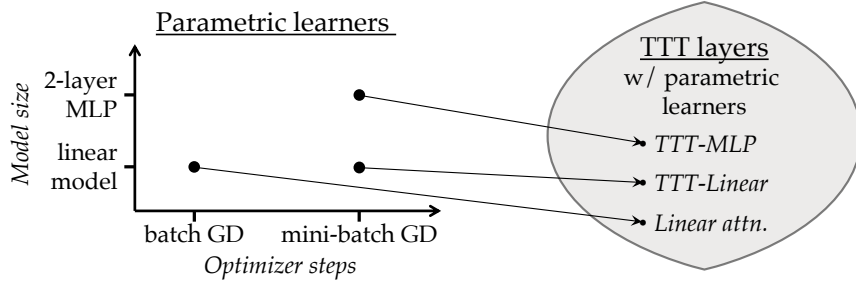


图 8. 参数化学习器需要定义两个属性：模型和优化器（左），每个学习器唯一地诱导出一个 TTT 层（右）。本文提出了两种诱导出的 TTT 层：TTT-线性和 TTT-多层感知机（MLP）。具有线性模型和批量梯度下降（GD）的 TTT 层等价于线性注意力 [44]。

So W_b 可以方便地通过矩阵乘法计算。为计算 $Z = [z_1, \dots, z_b]$ ，已知：

$$z_t = f(x_t; W_t) = W_t x_t = \left(W_0 - \eta \sum_{s=1}^t G_t \right) x_t = W_0 x_t - 2\eta \sum_{s=1}^t (W_0 x_s - x_s) x_s^T x_t. \quad (7)$$

Denote $\delta_t = \sum_{s=1}^t (W_0 x_s - x_s) x_s^T x_t$ and the matrix $\Delta = [\delta_1, \dots, \delta_b]$. We can derive that:

$$\Delta = (W_0 X - X) \text{mask}(X^T X), \quad (8)$$

其中掩码为上三角掩码，其值为零（类似于注意力掩码，但用零代替无穷大），且项 $W_0 X - X$ 可从 W_b 的计算中复用。现在 Δ 也可方便地通过矩阵乘法计算。将 Δ_{back} 代入方程 7，我们得到 $Z = W_0 X - 2\eta \Delta$ 。

我们将此过程称为对偶形式，与之相对的是本小节之前的原始形式，其中 G 和 W 被显式物化。如前所述，两种形式在输出上是等价的。原始与对偶的术语沿用了先前在 TTT 之外探索类似数学表述的工作 [35, 7, 61]。在附录 A 中，我们证明当 f 为具有非线性层的神经网络时，对偶形式仍然有效，只是符号更为复杂。

TTT 小批量内原始形式的时间复杂度为 $O(b \times d^2)$ 。对偶形式的时间复杂度为 $O(b \times d^2)$ ，仅用于计算 W_b ，随后额外需要 $O(b^2 \times d)$ 用于计算 $z_1, \dots, z_b$ 。与原始形式相比，对偶形式牺牲了理论复杂度以换取硬件利用率。在实践中， d 通常为几百，而 b 仅选择为 16。因此，计算 $z_1, \dots, z_b$ 的墙钟时间相对较小，如图 7 右面板所示。在我们的 JAX 实现中，使用对偶形式进行训练比原始形式快 $5 \times$ 倍以上。

2.6 理论等价性

在 2.1 小节中，我们提到 f 可以是线性模型或神经网络。在 2.4 小节中，我们还讨论了更新规则的三种变体：在线梯度下降、批量梯度下降和小批量梯度下降。这 2×3 种组合中的每一种都诱导出 TTT 层的不同实例化，如图 8 所示。

我们现在证明，在这些诱导的实例化中，具有线性模型和批量梯度下降的 TTT 层等价于线性注意力 [44]，这是一种广为人知的 RNN 层。⁷ 简而言之，线性注意力 [44] 就是没有 Softmax 的自注意力。回顾自注意力的定义： $z_t = V_t \text{softmax}(\frac{K_t^T q_t}{\sqrt{d}})$ 。没有 Softmax 后，它变为 $z_t = V_t (K_t^T q_t) = \sum_{s=1}^t v_s k_s^T q_t$ ，这是线性注意力最简单的表述。与其他 RNN 层类似，它可以写成循环形式，其中 $\sum_{s=1}^t v_s k_s^T$ 是隐藏状态。由于 $\sum_{s=1}^t v_s k_s^T$ 可以通过对每个 $t = 1, \dots, T$ 进行累加求和来计算，线性注意力相对于 T 也具有线性复杂度。

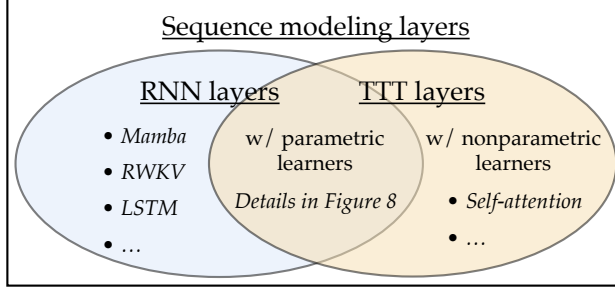


图 9. 循环神经网络层与测试时训练层均为序列建模层的子集。循环神经网络层具有一个在时间上大小固定的隐藏状态。采用参数化学习器的测试时训练层也属于循环神经网络层，因为其隐藏状态同样大小固定。采用非参数化学习器的测试时训练层可以表示自注意力，如第 2.6 小节所述。

定理 1. 考虑以 $f(x) = Wx$ 为内环模型、以批量梯度下降 $\eta = 1/2$ 为更新规则、且 $W_0 = 0$ 的测试时训练层。那么，给定相同的输入序列 $x_1, \dots, x_T$ ，方程 5 中定义的输出规则产生与线性注意力相同的输出序列 $z_1, \dots, z_T$ 。

证明. 根据方程 4 中 ℓ 的定义， $\nabla \ell(W_0; x_t) = -2(\theta_V x_t)(\theta_K x_t)^T$ 。根据方程 6 中批量梯度下降的定义：

$$W_t = W_{t-1} - \eta \nabla \ell(W_0; x_t) = W_0 - \eta \sum_{s=1}^t \nabla \ell(W_0; x_s) = \sum_{s=1}^t (\theta_V x_s)(\theta_K x_s)^T.$$

将 W_t 代入方程 5 中的输出规则，我们得到输出令牌：

$$z_t = f(\theta_Q x_t; W_t) = \sum_{s=1}^t (\theta_V x_s)(\theta_K x_s)^T (\theta_Q x_t),$$

这正是线性注意力的定义。 □

在表 1 中，我们首先通过改进的线性注意力实现来实证验证上述等价性。⁸ 然后，为了说明每个组件的贡献（包括下一小节将介绍的一些组件），我们将它们逐行添加到与线性注意力等价的测试时训练层中，最终得到我们提出的实例化方法，称为测试时训练线性层。从批量梯度下降改为小批量梯度下降的改进幅度最大。

虽然图 8 中的模型 $\times$ 优化器空间已经很大，但机器学习远比优化模型 f 的参数 W_t 丰富。还有非参数化学习器，如最近邻、支持向量机和核岭回归。根据定义，非参数化学习器没有参数 W_t ，而是直接使用训练数据 $x_1, \dots, x_t$ 。因此我们使用记号 $f(x; x_1, \dots, x_t)$ 。我们现在证明，对于特定的非参数化学习器，所诱导的测试时训练层等价于自注意力。

定理 2. 考虑具有纳达拉亚-沃森 (Nadaraya-Watson) 估计器 [6, 11] 的测试时训练 (TTT) 层，其定义为：

$$f(x; x_1, \dots, x_t) = \frac{1}{\sum_{s=1}^t \kappa(x, x_s)} \sum_{s=1}^t \kappa(x, x_s) y_s, \quad (9)$$

其中 $y_s = \theta_V x_s$ 是第 2.3 小节中讨论的标签视图，并且

$$\kappa(x, x'; \theta_K, \theta_Q) \propto e^{(\theta_K x)^T \theta_Q x'} \quad (10)$$

是一个具有带宽超参数 θ_K 和 θ_Q 的核函数。那么，给定相同的输入序列 $x_1, \dots, x_T$ ，方程 5 中定义的输出规则会产生与自注意力相同的输出序列 $z_1, \dots, z_T$ 。

⁸ 线性注意力在 [44] 中的原始公式包含一个归一化器和对 x_t 的特征扩展，这些仍然可以包含在等价的测试时训练 (TTT) 层中。然而，先前的工作发现这两项添加可能会损害性能 [60]，我们在自己的实验中也验证了这一点（表 1 的第一行与第二行）。因此，我们仅构建一个与不含这两项添加的最简线性注意力公式等价的测试时训练 (TTT) 层。

配置	困惑度	差异
线性注意力 [44]	15.91	- 改进的线性注意力
15.23	-0.68	测试时训练 (TTT) 等价
15.23 0 + 可学习		
W_0	15.27	+0.04
+ LN and residual in f	14.05	-1.22
+ 小批量测试时训练 (TTT)	12.35	-1.70
+ 可学习 η	11.99	-0.36
+ Mamba backbone	11.09	-0.90

表 1. 从线性注意力改进的消融实验。此处所有模型均具有 1.25 亿参数，并按照第 3.1 小节中的方案进行训练。最后一行（困惑度为 11.09）是图 10 中测试时训练-线性 (TTT-Linear) 的最终结果。从第 2.6 小节讨论的等价性出发，可学习的 W_0 会略微损害性能，但下面的各行在没有它的情况下无法稳定训练。最大的改进来自小批量测试时训练 (TTT)（从 $b = T = 2048$ 变为 $b = 16$ ）。其次来自层归一化 (LN) 和残差连接实例化内部模型 f 。如果没有测试时训练 (TTT) 的概念框架，这两种设计都很难被发现。

证明。将上述 $\mathcal{V}_s$ 和 κ 代入方程 9，即可得到自注意力的定义。□

附录 B 包含了对上述 Nadaraya-Watson 估计量与核函数 κ 的详细说明。与定理 1 相比，定理 2 并未产生不同于注意力机制的实现。

对于上述的测试时训练层，隐藏状态是 $x_1, \dots, x_t$ 或类似的处理后训练数据列表，更新规则将 x_t 添加到列表中，输出规则用 κ 扫描列表。在前面的小节中，我们的隐藏状态被定义为 W_t ，更新规则是一个梯度步骤，输出规则是对 f 的调用。为了统一这两种构造，我们定义了一种称为学习器的新抽象，它唯一地诱导出一个测试时训练层。

与标准机器学习包 [57] 中的定义类似，所有学习器都需要实现两个方法：训练和预测。现在我们将诱导的测试时训练层的隐藏状态重新定义为学习器的内部存储，并将更新和输出规则分别定义为训练和预测方法。在测试时训练层的这一新定义下，定理 1 中的参数化学习器和定理 2 中的非参数化学习器均可被涵盖。图 9 在更广泛的序列建模层范围内总结了测试时训练层的这一通用定义。

这一通用定义对参数化学习器还有额外益处：参数化学习器的内部存储中除 W 外还可包含更多对象，例如优化器状态，这些也将被纳入所诱导的 TTT 层的隐藏状态中。这一扩展使得 TTT 层在未来工作中能够使用更复杂的优化器，如 Adam [45]。

2.7 实现细节

实例化 f . 本文提出两种变换器 (Transformer) 测试时训练 (TTT) 层变体——TTT-线性 (TTT-Linear) 与 TTT-多层感知机 (TTT-MLP)，二者仅在 f 的实例化方式上有所不同。对于 TTT-线性， $f_{\text{lin}}(x) = Wx$ ，其中 W 为方阵。对于 TTT-多层感知机， f_{MLP} 具有两层结构，类似于变换器中的多层感知机。具体而言，隐藏层维度为输入维度的 $4 \times$ 倍，其后接 GELU 激活函数 [31]。为提升测试时训练过程中的稳定性， f 始终包含层归一化 (Layer Normalization, LN) 与残差连接。即， $f(x) = x + \text{LN}(f_{\text{res}}(x))$ ，其中 f_{res} 可为 f_{lin} 或 f_{MLP} 。

可学习性 W_0 ·TTT 初始化 W_0 在所有序列间共享，尽管后续权重 $W_1, \dots, W_T$ 对于每个输入序列是不同的。我们不在外循环中将 $W_0 =$ 设为 0，而是将其作为外循环的一部分进行学习。由于外循环参数始终用 θ 表示而非 W ，我们为其分配别名 $\theta_{\text{init}} = W_0$ 。实际上，与重建视图 θ_K 、 θ_Q 、 θ_V 相比， θ_{init} 增加的数量可忽略不计，因为其输入和输出都是低维的。经验上，我们观察到学习 W_0 能显著提升训练稳定性。

可学习性 η ·学习率通常是梯度下降中最重要的超参数，因此我们尝试将方程 6 中的内循环学习率 η 作为外循环的一部分进行学习。为使灵活性更强，我们令 η 成为输入令牌的函数（因此随时间变化）。具体设计为 $\eta(x) = \eta_{\text{base}} \sigma(\theta_{\text{lr}} \cdot x)$ ，其中可学习向量 θ_{lr} 是外循环参数， σ 是 Sigmoid 函数，标量 η_{base} 是基础学习率，TTT-Linear 设为 1，TTT-MLP 设为 0.1。或者， $\eta(x)$ 也可解释为 $\nabla \ell$ 的门控。

骨干架构。将任意 RNN 层集成到更大架构中最直接的方法是直接替换 Transformer 中的自注意力，在此上下文中称为骨干。然而，现有的 RNN 如 Mamba[27] 和 Griffin[18] 都使用与 Transformer 不同的骨干。最值得注意的是，它们的骨干在 RNN 层之前包含时间卷积，这可能有助于收集跨时间的局部信息。在实验了 Mamba 骨干后，我们发现它也能提升 TTT 层的困惑度，因此将其纳入我们提出的方法中。详见附录中的图 13。

3 实验

我们通过与两个基线——Transformer 和现代 RNN Mamba——进行比较来评估 TTT-Linear 和 TTT-MLP。我们的主要代码库基于 EasyLM[25]，这是一个用于在 JAX 中训练和部署 LLM 的开源项目。所有实验均可使用第一页底部提供的公开代码和数据集复现。

数据集。遵循曼巴（Mamba）论文 [27]，我们在堆（Pile）数据集 [24] 上进行了 2k 和 8k 上下文长度的标准实验，该数据集是训练开源大语言模型（LLM）[8] 的常用文档集合。然而，堆数据集中长度超过 8k 的序列很少 [19]。为了评估长上下文能力，我们还在堆数据集的一个子集——书籍 3（Books3）上进行了实验，上下文长度从 1k 到 32k，以 $2 \times$ 为增量，该子集已被广泛用于长上下文大语言模型的训练 [52, 3]。

骨干架构。如第 2.7 小节所述，变换器（Transformer）和曼巴使用不同的骨干网络，而 TTT 线性（TTT-Linear）和 TTT 多层感知机（TTT-MLP）除非特别说明，始终使用曼巴骨干网络。作为消融实验，图 10 和图 11 展示了变换器骨干网络中的 TTT 层。当图中同时包含变换器骨干网络和曼巴骨干网络时，我们分别用（T）和（M）表示。

协议。为确保对基线的公平性，我们尽可能严格遵循曼巴论文中的评估协议：

- 对于每种评估设置（例如数据集、上下文长度和方法），我们实验了四种模型规模：125M、350M、760M 和 1.3B 参数。对于曼巴，相应规模为 130M、370M、790M 和 1.4B，因为曼巴不遵循变换器配置。
- 所有模型均采用曼巴论文中描述并在附录 C 中复现的金吉拉（Chinchilla）训练方案 9 进行训练。我们的变换器基线基于羊驼（Llama）架构 [75]，也遵循曼巴论文中的基线。作为验证，我们的基线能够复现曼巴论文在其评估设置中报告的结果。¹⁰

⁹ 金吉拉论文是另一项关于经验缩放定律 [34] 的极具影响力的研究。通过大量超参数的大规模实验，他们观察到计算最优模型遵循特定的训练方案。我们仅遵循曼巴论文中使用的金吉拉方案，该方案可能与 [34] 中的原始方案略有不同。¹⁰ 我们的协议与曼巴论文的唯一区别在于分词器。曼巴论文在不同实验中使用了两种分词器——GPT-2 和 GPT-NeoX。为保持一致性，本文全文采用单一分词器，并选择羊驼分词器 [75]，这是现代最先进的分词器。

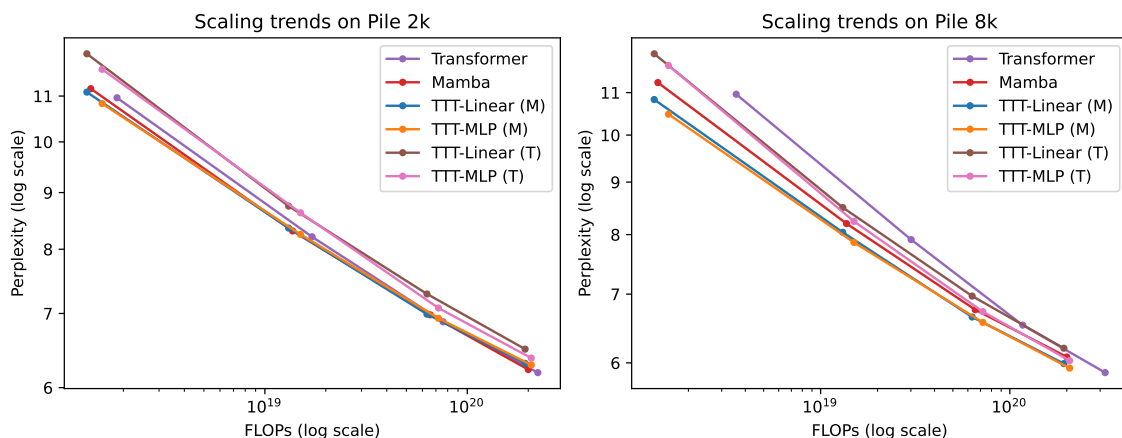


图 10. 在堆数据集上对 2k 和 8k 上下文长度的评估。细节见第 3.1 小节。TTT 线性在 2k 上下文下性能与曼巴相当，在 8k 上下文下性能更优。

- 本文未对混合架构（如格里芬（Griffin）[18]）进行实验，因为基线并非混合架构。尽管同时使用自注意力与测试时训练（TTT）层的混合架构可能提升性能，但会降低学术评估的清晰度。

3.1 短上下文：Pile 数据集

从图 10 中，我们得出以下几点观察：

- 在 2k 上下文长度下，TTT 线性（M）、曼巴（Mamba）与变换器（Transformer）的性能相当，曲线大部分重叠。在较大浮点运算量预算下，TTT 多层感知机（M）表现略差。尽管在每个模型规模上 TTT 多层感知机的困惑度优于 TTT 线性，但额外的浮点运算成本抵消了这一优势。

- 在 8k 上下文长度下，TTT 线性（M）和 TTT 多层感知机（M）的表现均显著优于曼巴（Mamba），这与 2k 上下文长度下的观察结果形成对比。甚至采用变换器（Transformer）骨干的 TTT 多层感知机（T）在约 13 亿参数规模下也略优于曼巴。本文中观察到的一个稳健现象是：随着上下文长度的增加，TTT 层相对于曼巴的优势不断扩大。

- 在 8 千上下文长度下，变换器（Transformer）在每个模型规模上仍具有良好（即便不是最佳）的困惑度，但由于浮点运算次数（FLOPs）成本较高，其曲线并不具备竞争力。

E 骨干网络的影响。将 TTT 层从曼巴骨干网络切换到变换器骨干网络会产生两个效果。首先，在我们目前的评估中，采用曼巴骨干网络的 TTT 层表现更佳。其次，采用曼巴骨干网络时，TTT-MLP 最多只能与 TTT-Linear 相当；但采用变换器骨干网络时，TTT-MLP 明显更优。我们假设，当序列建模层的隐藏状态表达能力较弱时，曼巴骨干网络中的时间卷积能提供更多帮助。线性模型比 MLP 表达能力弱，因此从卷积中获益更多。我们将在下一小节重新审视这一假设。

缺乏线性拟合。Chinchilla 论文通过实验观察到，按照其配方得到的计算最优模型在浮点运算次数与困惑度的双对数图中落在一条直线上，这在缩放定律实验中是常见现象 [34]。然而，在图 10 中我们并未观察到清晰的线性拟合。

图 11（书籍中的类似实验），甚至对于变换器（Transformer）也是如此。这并不令人意外

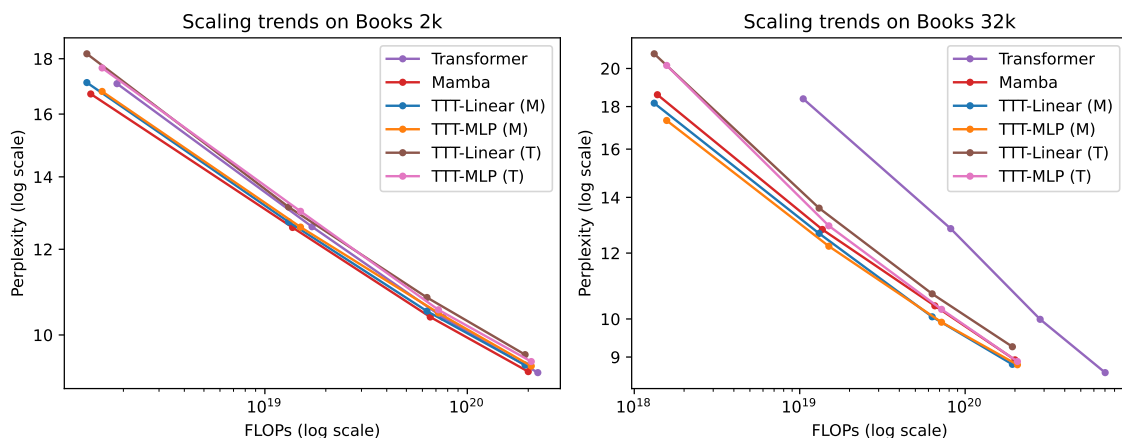


图 11. 在 Books 数据集上对上下文长度 2k 和 32k 的评估。细节见第 3.2 小节。我们在上下文长度 1k、2k、4k、8k、16k、32k 上的完整结果，包括变换器微调，见图 15（附录）。来自 Pile 数据集的大部分观察结果仍然成立。

鉴于数据集、上下文长度、分词器和架构的差异。遵循曼巴（Mamba）论文的做法，由于误差较大，我们采用连线方式连接数据点，而非使用线性回归进行拟合。¹¹

3.2 长上下文：书籍

为了评估长上下文能力，我们在 $2 \times$ 的增量下，使用 Pile 的一个流行子集 Books3，实验了从 1k 到 32k 的上下文长度。这里的训练方案与 Pile 相同。¹² 从图 11 中的部分结果，我们得出以下几点观察：• 在 Books 的 2k 上下文上，Pile 2k 的所有观察仍然成立，除了 Mamba 现在比 TTT-Linear 表现略好（而在 Pile 2k 中它们的线条大致重叠）。

- 在 32k 上下文长度下，TTT 线性（M）和 TTT 多层感知机（M）的表现均优于曼巴（Mamba），这与 Pile 8k 上的观察结果相似。甚至采用变换器（Transformer）骨干的 TTT 多层感知机（T）在 32k 上下文长度下也略优于曼巴。

- TTT-MLP（T）在 13 亿参数规模下仅略逊于 TTT-MLP（M）。如前所述，由于缺乏清晰的线性拟合，难以推导出经验缩放定律。然而，TTT-MLP（T）的强劲趋势表明，变换器（Transformer）骨干网络可能更适合于超出我们评估范围的更大模型和更长上下文。

由于训练大语言模型的成本限制，本文仅对 2k 和 32k 两种长度的主干网络进行了消融实验。对于未来工作，本文认为，若测试时训练层具备更强表达能力的隐藏状态，则带有时间卷积的曼巴主干网络将不再必要。

变换器微调。虽然我们一直按照曼巴论文从头训练变换器，但在实践中，这种方法很少用于长上下文。标准做法是先在短上下文上训练变换器，然后在长上下文上进行微调。为了反映这一做法，我们增加了 11。理想情况下，我们会根据我们的评估设置，遵循钦奇拉论文中的流程，重新运行所有超参数并为每种方法推导出可能的新配方。如果新的计算最优模型确实落在一条直线上，我们就能预测当前浮点运算次数范围之外的性能 [43, 34]。然而，这项实证研究所需的资源将比我们的多出几个数量级。¹² 按照曼巴论文的做法，无论上下文长度如何，我们始终使用每个训练批次 50 万标记。换句话说，对于上下文长度 T ，每个批次有 $0.5M/T$ 个序列（假设可整除）。

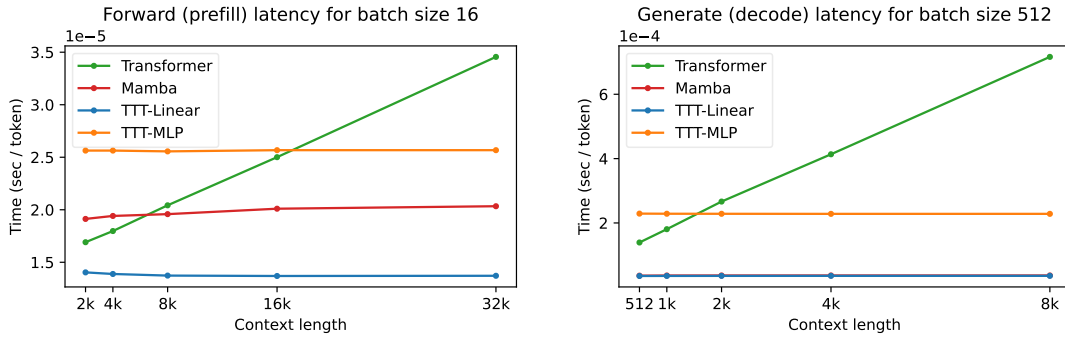


图 12. 在配备 80G HBM 和 PCIe 连接的 NVIDIA A100 GPU 上的延迟。

另一个基线，TF 微调，适用于上下文长度 4k 及以上。该基线从根据 Chinchilla 配方在 Books 2k 上训练的模型开始，然后使用额外 20% 的标记在指定上下文长度下进行微调，遵循 Llama Long 论文 [81]。TF 微调配方的详细信息见附录 C。

图实验 图 2 (右)。与 TTT-Linear 相比，在匹配浮点运算次数 (FLOPs) 的情况下，TTT-MLP 在短上下文上表现较差，但在长上下文上表现更好。这一观察结果符合我们的预期，即作为隐藏状态的多层感知机 (MLP) 比线性模型更具表达力：更具表达力的隐藏状态其较大容量在长上下文中得到充分利用 (因此是优势)，但在短上下文中则显得冗余 (因此在匹配 FLOPs 的设置下是劣势)。本图中的 Transformer 是 TF 微调 (TF finetune)，它是 **32k** 上下文下更强的基线。图 2 实验的详细信息见附录 C。

我们在上下文长度 1k、2k、4k、8k、16k、32k 下的完整结果，包括 TF 微调，见图 15 (见附录)。

3.3 实际运行时间

大语言模型 (LLM) 的训练和推理可以分解为前向、反向和生成。推理过程中的提示处理，也称为预填充，与训练中的前向操作相同，只是中间激活不需要为反向存储。由于前向 (训练和推理期间) 和反向都可以并行化，我们使用对偶形式。生成新标记，也称为解码，本质上是顺序的，因此我们使用原始形式。

由于资源限制，我们的实验使用 JAX 编写并在 TPU 上运行。在 v5e-256 TPU pod 上，变换器 (Transformer) 基线在上下文长度为 2k 时每次训练迭代耗时 0.30 秒，而 TTT-Linear 耗时 0.27 秒，在没有任何系统优化的情况下已经快了 10%。然而，Mamba (使用 PyTorch、Triton 和 CUDA 实现) 只能在 GPU 上运行，因此为了公平比较，我们也将我们的方法重写为 GPU 内核。由于训练内核需要大量工作且无法在我们的 TPU 上使用，本文仅编写推理内核。

图 12 展示了我们推理内核在前向 (预填充) 和生成 (解码) 中的延迟。所有模型均为 1.3B (Mamba 为 1.4B)。正如预期，变换器的每标记时间随上下文长度线性增长，而其他方法大致保持不变。¹³ 注意我们的

¹³ 我们观察到 TTT-Linear、TTT-MLP 和 Mamba 的网络前向延迟略有增加，尽管每个序列建模层本身的延迟保持不变。考虑操作 θX ，其中 θ 是 $d \times d$ ， X 是 $d \times T$ 。其延迟 (按 T 归一化) 预期为常数，但实际上随 T 略有增长。造成这种现象的一个可能原因是当 T 变得非常大时 GPU 降频 [30]。

变换器基线明显快于 Mamba 论文中的速度，因为我们使用了最先进的服务系统 vLLM[49]，而不是 HuggingFace Transformer[80]。

4 相关工作

4.1 测试时学习

测试时学习的思想在机器学习中有悠久的历史。该思想最早的版本之一被称为局部学习（Local Learning）（Bottou 和 Vapnik [9]）：对于每个测试输入，在做出预测之前，先在其邻居上进行训练。这一过程已被有效地应用于从支持向量机（SVM）[85] 到现代大语言模型（LLM）[29] 的各种模型。

测试时学习的另一个早期版本被称为直推学习（Transductive Learning）[22]。正如 Vladimir Vapnik [76] 所述，直推的原则是“……得到你真正需要的答案，而不是一个更一般的答案。”直推学习的实用实现利用测试数据为支持向量机的间隔添加约束 [42, 16]。然而，直推学习通常需要多个测试实例才能在经验上有效，这与许多测试时训练的实例化不同，后者一次只需要一个测试实例（图像、视频或自然语言序列）。

在计算机视觉中，测试时学习的思想几十年来已被应用于人脸检测 [41]、目标检测 [56]、图像超分辨率 [68] 和三维重建 [53] 等应用。最近，同样的思想也被应用于自然语言处理，被称为动态评估（Dynamic Evaluation）[47, 48]。其基本方法是直接在测试序列上微调语言模型，测试序列通常以提示的形式出现。

接下来，我们详细讨论两条相关的工作线：测试时训练和快速权重。

4.1.1 测试时训练

测试时训练（Test-Time Training, TTT）的核心思想是，每个测试实例都定义了自己的学习问题，其中该测试实例本身就是泛化的目标 [72]。具体来说，对于每个测试实例 x ，传统做法是使用针对所有训练实例平均优化的预测器 f 进行预测 $f(x)$ 。TTT 首先制定一个由 x 定义的学习问题，然后在 x 上训练一个模型 f_x （通常以 f 作为初始化），并进行预测 $f_x(x)$ 。

由于测试实例没有标签，学习问题只能通过自监督任务来构建。先前的研究表明，基于重建的测试时训练（TTT）显著提升了性能，尤其是在异常值上 [23]。当测试对象是流式到达的视频帧且测试时训练采用自回归方式时，改进更为显著 [79]，因为 f_t 是在过去的帧上训练的。自回归连接使得 [79] 与本文最为相关。

$x_1, \dots, x_t$

从概念上讲，本文与先前工作最大的区别在于，我们的重建任务是在外循环中学习得到的，而非基于人类先验手工设计。后续工作探索了机器人操作 [28] 和运动控制 [71] 等应用，这些应用通常需要为自监督任务设计不同的方案。在一份初步手稿 [70] 中，我们从基于重建的测试时训练视角出发，探索了“在测试时学习”这一思想，并在物体识别任务上进行了实验。

4.1.2 快速权重与快速权重编程器

快速权重的一般思想是仅在最相关的数据上更新“快速”模型的参数，这与在所有数据上更新“慢速”模型的传统做法不同 [74]。这一思想自 20 世纪 80 年代就已存在 [20, 32, 78]。最相关的数据通常包括测试实例本身，因此测试时训练可以视为快速权重的一种特例。与快速权重相比，

测试时训练倡导构建一个明确的学习问题，其中测试实例是泛化的目标。我们的更新规则也是一个明确的优化步骤。

快速权重编程器（FWP）的思想是在测试时用一个更新频率较低甚至不更新的“慢速”模型来更新快速权重 [65]。我们的内循环权重 W 可以视为“快速”权重，外循环权重 θ 可以视为“慢速”权重。因此，包含测试时训练层的网络可以视为快速权重编程器的一种特例 [46]，类似于测试时训练可以视为快速权重的一种特例。值得注意的是，Irie 等人 [38] 中的一种实例化将快速权重实现为多层感知机，这早于我们的测试时训练多层感知机。

许多其他现代循环神经网络层，如线性注意力（Linear Attention）[44, 63] 和 DeltaNet [62, 83]，也受到快速权重程序（FWP）思想的启发。鉴于它们与本文工作的相关性，我们将在下一小节详细讨论这些现代循环神经网络层。在本小节剩余部分，我们简要概述其他几种形式的快速权重程序。Clark 等人 [15] 为变换器（Transformer）添加了最后一层快速权重，其初始化被训练为慢权重。Irie 等人 [39] 设计了由自身编程的快速权重，这可以解释为递归自我改进。此外，[40] 利用图像作为快速权重构建图像生成器，[37] 将快速权重程序的连续时间扩展应用于时间序列分类，而 [36] 和 [26] 则展示了更新规则的选择如何影响快速权重程序在形式语言识别任务上的表达能力。

4.2 现代循环神经网络层

本文的基线 Mamba [27] 只是众多继承线性注意力（Linear Attention）[44, 63] 的线性（矩阵）隐藏状态的近期循环神经网络层之一。一些更新的例子包括 RWKV [58, 59]、xLSTM [4] 和门控线性注意力（Gated Linear Attention, GLA）[82]。最相关的工作是 DeltaNet [62]，它等价于内循环小批量大小为 1 且没有层归一化和残差连接的 TTT-Linear。在 [83] 中，Yang 等人进一步提升了 DeltaNet 的性能，并实现了跨令牌（在本文术语中，即跨内循环小批量）的并行更新。自本文第一版发布以来，具有矩阵（线性）隐藏状态的循环神经网络层在 Mamba 2 [17] 和门控 DeltaNet [82] 中也得到了进一步发展。

与这一系列工作相比，本文的贡献在于提供了一个实用的框架，可以将任意神经网络实例化为隐藏状态。然而，即使应用了本文在效率方面的改进，此类实例化仍可能需要大量的实际运行时间。例如，如图 2 所示，TTT-MLP 在浮点运算次数（FLOPs）方面是有效的。但如图 12 所示，多层感知机（MLP）结构带来的额外复杂度使得实际运行时间相对于浮点运算次数的增加要大得多。本文框架能否产生克服这一局限性或带来更大收益的实例化，仍有待观察。

4.3 学会学习

数十年来，研究人员一直主张学会学习（Learning to Learn），亦称元学习（Meta-Learning）或双层优化（Bi-level Optimization），应作为智能的关键组成部分 [64, 5, 73, 50]。在 [2]、[21] 和 [55] 等先前工作中，内层循环每次从整个数据集学习，而非从序列学习，因此外层循环需要数据集或任务的集合。简言之，外层循环比常规训练“高一个层级”。由于难以收集数百万个数据集，该外层循环难以扩展。

相比之下，对于测试时训练（TTT），每个序列本身即是一个数据集，并定义其自身的泛化问题。内层循环比常规训练“低一个层级”，因此我们的外层循环只是监督学习（Supervised Learning）经典问题的另一种解决方案，而非跨数据集泛化这样的新问题设定。如表 2 所示，我们的外层循环与常规训练“处于同一层级”。这使得我们的外层循环更易于扩展。

	Inner loop	Outer loop	Subsection
Piece of data	Token x_t	Sequence $x_1, \dots, x_T$	2.1, 2.2
Training set	Sequence $x_1, \dots, x_T$	Dataset of sequences, e.g., Books	
Objective	Reconstruction (loss ℓ)	Next-token prediction	
Parameters	W (weights of f)	θ_{rest} (rest of the network)	
		$\theta_K, \theta_Q, \theta_V$ (reconstruction views)	2.3
		θ_{init} and θ_{lr}	2.7

表 2. 总之，本文将监督学习重新表述为具有两个嵌套循环的学会学习。外层循环的高亮行与常规训练相同。外层循环的参数成为内层循环的超参数。直观上，内层循环，即测试时训练，比常规训练“低一个层级”。

5 讨论

本文将监督学习的经典问题重新表述为在测试时学习（Learn at Test Time）。该表述为传统上称为网络架构（Network Architectures）的构建提供了一种替代概念框架。我们在表 2 中总结了当前的实例化。

未来工作。该框架内有效实例化的搜索空间巨大，本文仅迈出了一小步。幸运的是，若我们的视角成立，则常规训练中的启发式方法可迁移至测试时训练，且搜索可高效进行。接下来，我们概述一些尤其有前景的未来研究方向：

- **外层循环参数化。**对多视图重建任务族或更一般的自监督任务族进行参数化的方法还有很多。如果我们尝试的第一种方法恰好是最好的，那将是一个很大的巧合。

- **系统优化。**第 3.3 节中的系统优化充其量只是初步的，尚有许多改进途径。此外，通过时间的流水线并行可能使我们能够在多个设备上共同处理长达数百万词元的长序列。

- **更长的上下文与更大的模型。**受限于学术资源，本文未在数百万或数十亿的上下文长度上进行训练，而根据相关研究，这同样需要更大规模的模型

图 16. 测试时训练（TTT）层的优势在更长上下文中应更为显著。

- **更宏大的 f 实例化。**当上下文长度变长时， f 也需要更大。对于视频任务和具身智能体，其上下文长度很容易扩展到数百万甚至数十亿， f 可以是卷积神经网络。

- **多层次学会学习。**如果 f 本身是一个自注意力层，那么根据定理 2，它可以被解释为嵌套在现有内循环中的另一个内循环。通过这种方式，我们有可能构建多层嵌套的学习问题。这一思想的变体已在 [38] 和 [39] 中进行了探索。

我们为什么要研究测试时训练（Test Time Training, TTT）？首先是一个更基本的问题：为什么要研究人工智能？对我们中的一些人来说，人工智能是探索人类智能本质的游乐场。先前的工作通常尝试用机器学习来建模人类学习，其中训练是在具有独立同分布实例的打乱数据集上进行的，而推理则是在单独的测试集上进行的。然而，人类并不是自然地通过独立同分布实例来学习，也没有训练-测试的划分。我们相信，人类学习与 TTT（我们的内循环）有着更有前景的联系，其数据是一个可能非常长的、具有强时间依赖性的序列，任何数据片段都可以同时用于训练和测试。这就是我们研究 TTT 的原因。

作者贡献

Yu Sun started this project with Xinhao Li in November 2022, and has been working on it full-time since June 2023. Yu proposed the conceptual framework of the project, designed mini-batch TTT and the dual form, wrote the paper with help from others, and led the daily operations of the team.

Xinhao Li started this project with Yu Sun in November 2022, and has been working on it full-time since then. Xinhao and Karan co-led the development of our current codebase. Before March 2024, Xinhao was the primary contributor to our earlier codebases that shaped this project. Xinhao made significant contributions to the project direction in discussions.

Karan Dalal joined this project full-time in June 2023. In collaboration with Xinhao, he co-led the development of our current codebase. Karan managed the experiments in Section 3, helped write the paper, and made significant contributions to the project direction in discussions.

Jiarui Xu joined this project in March 2024. He led our architectural development since he joined, and made significant contributions to the project direction in discussions.

Arjun Vikram joined this project in September 2023. He made significant contributions to our systems optimization, as well as current and earlier codebases.

Genghan Zhang joined this project in January 2024. He provided critical insights and made significant improvements to our systems optimization.

Yann Dubois joined this project in February 2024. He proposed our current instantiation of f , and made significant contributions to the project direction in discussions.

Xinlei Chen and Xiaolong Wang have been supporting this project since November 2022, and the direction of test-time training for many years. Without their support in compute and organization, this project could not have survived its early stage. They gave invaluable advice to our experiments.

Sanmi Koyejo, Tatsunori Hashimoto, and Carlos Guestrin have been supporting this project since May 2023. They gave invaluable advice to our experiments and presentation. For example, Sanmi suggested us to focus on TTT-Linear, Tatsu suggested the experiments in Figure 2 (left), and Carlos outlined Section 2.

致谢

本项目部分计算资源由谷歌 TPU 研究云计划和 Hyperbolic Labs 慷慨支持。XW 部分得到了亚马逊研究奖、思科教师奖和高通创新奖学金的资助。SK 感谢 NSF 2046795 和 2205329、NIFA 奖项 2020-67021-32799、阿尔弗雷德·P·斯隆基金会以及谷歌公司的支持。TH 得到了索尼教师创新奖和松下公司的一份礼物的支持。CG 感谢空军科学研究办公室 (AFOSR) FA9550-20-1-0427、斯坦福以人为本人工智能 (HAI) 研究所以及谷歌和 IBM 的礼物支持。

我们要感谢 Rohan Taori、Xuechen Li、Allan Zhou、Ke Chen 和 Guandao Yang 的许多有益讨论，Menghao Guo 在代码发布方面的帮助，Xinyang Geng 在 EasyLM 方面的帮助，Hao Liu 在 LWM 代码库方面的帮助，David Hall 在 Levanter 方面的帮助，Yossi Gandelsman 和 Yutong Bai 在项目早期阶段的帮助，Mert Yuksekgonul 在论文图表方面的帮助，Horace He、Ben Spector 和 Azalia Mirhoseini 在系统方面的帮助，Sharad Vikram 和 Roy Frostig 回答我们关于 JAX 和 Pallas 的问题，Albert Gu 和 Tri Dao 帮助我们复现 Mamba 论文中的实验，以及 Kilian Weinberger 和 Percy Liang 在展示方面的建议。Yu Sun 感谢他的博士导师 Alexei A. Efros 和 Moritz Hardt 多年前的许多洞见，这些洞见最终成为了本文的一部分。

参考文献

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- [2] Marcin Andrychowicz, Misha Denil, Sergio Gomez, Matthew W Hoffman, David Pfau, Tom Schaul, Brendan Shillingford, and Nando De Freitas. Learning to learn by gradient descent by gradient descent. *Advances in neural information processing systems*, 29, 2016.
- [3] Authors Guild. You just found out your book was used to train ai. now what?, 2023. Accessed: 2024-06-24.
- [4] Maximilian Beck, Korbinian Pöppel, Markus Spanring, Andreas Auer, Oleksandra Prudnikova, Michael Kopp, Günter Klambauer, Johannes Brandstetter, and Sepp Hochreiter. xlstm: Extended long short-term memory. *arXiv preprint arXiv:2405.04517*, 2024.
- [5] Yoshua Bengio, Samy Bengio, and Jocelyn Cloutier. *Learning a synaptic learning rule*. Citeseer, 1990.
- [6] Hermanus Josephus Bierens. The nadaraya-watson kernel regression function estimator. (*Serie Research Memoranda; No. 1988-58*). *Faculty of Economics and Business Administration, Vrije Universiteit Amsterdam.*, 1988.
- [7] Christopher M Bishop and Nasser M Nasrabadi. *Pattern recognition and machine learning*, volume 4. Springer, 2006.
- [8] Sid Black, Stella Biderman, Eric Hallahan, Quentin Anthony, Leo Gao, Laurence Golding, Horace He, Connor Leahy, Kyle McDonell, Jason Phang, et al. Gpt-neox-20b: An open-source autoregressive language model. *arXiv preprint arXiv:2204.06745*, 2022.
- [9] Léon Bottou and Vladimir Vapnik. Local learning algorithms. *Neural computation*, 4(6):888–900, 1992.
- [10] Leo Breiman, William Meisel, and Edward Purcell. Variable kernel estimates of multivariate densities. *Technometrics*, 19(2):135–144, 1977.
- [11] Zongwu Cai. Weighted nadaraya–watson regression estimation. *Statistics & probability letters*, 51(3):307–318, 2001.
- [12] Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. Training deep nets with sublinear memory cost, 2016.
- [13] Xinlei Chen, Haoqi Fan, Ross Girshick, and Kaiming He. Improved baselines with momentum contrastive learning. *arXiv preprint arXiv:2003.04297*, 2020.
- [14] Yen-Chi Chen. A tutorial on kernel density estimation and recent advances. *Biostatistics & Epidemiology*, 1(1):161–187, 2017.
- [15] Kevin Clark, Kelvin Guu, Ming-Wei Chang, Panupong Pasupat, Geoffrey Hinton, and Mohammad Norouzi. Meta-learning fast weight language models. *arXiv preprint arXiv:2212.02475*, 2022.
- [16] Ronan Collobert, Fabian Sinz, Jason Weston, Léon Bottou, and Thorsten Joachims. Large scale transductive svms. *Journal of Machine Learning Research*, 7(8), 2006.
- [17] Tri Dao and Albert Gu. Transformers are ssms: Generalized models and efficient algorithms through structured state space duality. *arXiv preprint arXiv:2405.21060*, 2024.

- [18] Soham De, Samuel L Smith, Anushan Fernando, Aleksandar Botev, George Cristian-Muraru, Albert Gu, Ruba Haroun, Leonard Berrada, Yutian Chen, Srivatsan Srinivasan, et al. Griffin: Mixing gated linear recurrences with local attention for efficient language models. *arXiv preprint arXiv:2402.19427*, 2024.
- [19] Harm de Vries. In the long (context) run, 2023. Accessed: 2024-06-24.
- [20] Jerome A Feldman. Dynamic connections in neural networks. *Biological cybernetics*, 46(1):27–39, 1982.
- [21] Chelsea Finn, Pieter Abbeel, and Sergey Levine. Model-agnostic meta-learning for fast adaptation of deep networks. In *International conference on machine learning*, pages 1126–1135. PMLR, 2017.
- [22] A. Gammerman, V. Vovk, and V. Vapnik. Learning by transduction. In *In Uncertainty in Artificial Intelligence*, pages 148–155. Morgan Kaufmann, 1998.
- [23] Yossi Gandelsman, Yu Sun, Xinlei Chen, and Alexei A. Efros. Test-time training with masked autoencoders. *Advances in Neural Information Processing Systems*, 2022.
- [24] Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, Shawn Presser, and Connor Leahy. The pile: An 800gb dataset of diverse text for language modeling, 2020.
- [25] Xinyang Geng. EasyLM: A Simple And Scalable Training Framework for Large Language Models. <https://github.com/young-geng/EasyLM>, mar 2023. <https://github.com/young-geng/EasyLM>.
- [26] Riccardo Grazi, Julien Siems, Arber Zela, Jörg KH Franke, Frank Hutter, and Massimiliano Pontil. Unlocking state-tracking in linear rnns through negative eigenvalues. *International Conference on Learning Representations (ICLR)*, 2024.
- [27] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.
- [28] Nicklas Hansen, Rishabh Jangir, Yu Sun, Guillem Alenyà, Pieter Abbeel, Alexei A Efros, Lerrel Pinto, and Xiaolong Wang. Self-supervised policy adaptation during deployment. *arXiv preprint arXiv:2007.04309*, 2020.
- [29] Moritz Hardt and Yu Sun. Test-time training on nearest neighbors for large language models. *arXiv preprint arXiv:2305.18466*, 2023.
- [30] Horace He. Strangely, matrix multiplications on gpus run faster when given "predictable" data! [short], 2024. Accessed: 2024-06-30.
- [31] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (gelus). *arXiv preprint arXiv:1606.08415*, 2016.
- [32] Geoffrey E Hinton and David C Plaut. Using fast weights to deblur old memories. In *Proceedings of the ninth annual conference of the Cognitive Science Society*, pages 177–186, 1987.
- [33] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.
- [34] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, and Laurent Sifre. Training compute-optimal large language models, 2022.

- [35] Kazuki Irie, Róbert Csordás, and Jürgen Schmidhuber. The dual form of neural networks revisited: Connecting test time predictions to training patterns via spotlights of attention. In *International Conference on Machine Learning*, pages 9639–9659. PMLR, 2022.
- [36] Kazuki Irie, Róbert Csordás, and Jürgen Schmidhuber. Practical computational power of linear transformers and their recurrent and self-referential extensions. *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.
- [37] Kazuki Irie, Francesco Faccio, and Jürgen Schmidhuber. Neural differential equations for learning to program neural nets through continuous learning rules. *Advances in Neural Information Processing Systems*, 35:38614–38628, 2022.
- [38] Kazuki Irie, Imanol Schlag, Róbert Csordás, and Jürgen Schmidhuber. Going beyond linear transformers with recurrent fast weight programmers. *Advances in Neural Information Processing Systems*, 34:7703–7717, 2021.
- [39] Kazuki Irie, Imanol Schlag, Róbert Csordás, and Jürgen Schmidhuber. A modern self-referential weight matrix that learns to modify itself. In *International Conference on Machine Learning*, pages 9660–9677. PMLR, 2022.
- [40] Kazuki Irie and Jürgen Schmidhuber. Images as weight matrices: Sequential image generation through synaptic learning rules. *International Conference on Learning Representations (ICLR)*, 2022.
- [41] Vidit Jain and Erik Learned-Miller. Online domain adaptation of a pre-trained cascade of classifiers. In *CVPR 2011*, pages 577–584. IEEE, 2011.
- [42] Thorsten Joachims. *Learning to classify text using support vector machines*, volume 668. Springer Science & Business Media, 2002.
- [43] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.
- [44] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are rnns: Fast autoregressive transformers with linear attention. In *International conference on machine learning*, pages 5156–5165. PMLR, 2020.
- [45] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- [46] Louis Kirsch and Jürgen Schmidhuber. Meta learning backpropagation and improving it. *Advances in Neural Information Processing Systems*, 34:14122–14134, 2021.
- [47] Ben Krause, Emmanuel Kahembwe, Iain Murray, and Steve Renals. Dynamic evaluation of neural sequence models. In *International Conference on Machine Learning*, pages 2766–2775. PMLR, 2018.
- [48] Ben Krause, Emmanuel Kahembwe, Iain Murray, and Steve Renals. Dynamic evaluation of transformer language models. *arXiv preprint arXiv:1904.08378*, 2019.
- [49] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 611–626, 2023.
- [50] Brenden M Lake, Tomer D Ullman, Joshua B Tenenbaum, and Samuel J Gershman. Building machines that learn and think like people. *Behavioral and brain sciences*, 40:e253, 2017.

- [51] Quoc V Le. Building high-level features using large scale unsupervised learning. In *2013 IEEE international conference on acoustics, speech and signal processing*, pages 8595–8598. IEEE, 2013.
- [52] Hao Liu, Wilson Yan, Matei Zaharia, and Pieter Abbeel. World model on million-length video and language with blockwise ringattention. *arXiv preprint arXiv:2402.08268*, 2024.
- [53] Xuan Luo, Jia-Bin Huang, Richard Szeliski, Kevin Matzen, and Johannes Kopf. Consistent video depth estimation. *ACM Transactions on Graphics (ToG)*, 39(4):71–1, 2020.
- [54] Dougal Maclaurin, David Duvenaud, and Ryan Adams. Gradient-based hyperparameter optimization through reversible learning. In *International conference on machine learning*, pages 2113–2122. PMLR, 2015.
- [55] Luke Metz, Niru Maheswaranathan, Brian Cheung, and Jascha Sohl-Dickstein. Meta-learning update rules for unsupervised representation learning. *arXiv preprint arXiv:1804.00222*, 2018.
- [56] Ravi Teja Mullapudi, Steven Chen, Keyi Zhang, Deva Ramanan, and Kayvon Fatahalian. Online model distillation for efficient video inference. *arXiv preprint arXiv:1812.02699*, 2018.
- [57] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [58] Bo Peng, Eric Alcaide, Quentin Anthony, Alon Albalak, Samuel Arcadinho, Stella Biderman, Huanqi Cao, Xin Cheng, Michael Chung, Matteo Grella, et al. Rwkv: Reinventing rnns for the transformer era. *arXiv preprint arXiv:2305.13048*, 2023.
- [59] Bo Peng, Daniel Goldstein, Quentin Anthony, Alon Albalak, Eric Alcaide, Stella Biderman, Eugene Cheah, Teddy Ferdinan, Haowen Hou, Przemysław Kazienko, et al. Eagle and finch: Rwkv with matrix-valued states and dynamic recurrence. *arXiv preprint arXiv:2404.05892*, 2024.
- [60] Zhen Qin, Xiaodong Han, Weixuan Sun, Dongxu Li, Lingpeng Kong, Nick Barnes, and Yiran Zhong. The devil in linear transformer. *arXiv preprint arXiv:2210.10340*, 2022.
- [61] Frank Rosenblatt. The perceptron: a probabilistic model for information storage and organization in the brain. *Psychological review*, 65(6):386, 1958.
- [62] Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. Linear transformers are secretly fast weight programmers. In *International Conference on Machine Learning*, pages 9355–9366. PMLR, 2021.
- [63] Imanol Schlag, Tsendsuren Munkhdalai, and Jürgen Schmidhuber. Learning associative inference using fast weight memory. *arXiv preprint arXiv:2011.07831*, 2020.
- [64] Jürgen Schmidhuber. *Evolutionary principles in self-referential learning, or on learning how to learn: the meta-meta-... hook*. PhD thesis, Technische Universität München, 1987.
- [65] Jürgen Schmidhuber. Learning to control fast-weight memories: An alternative to dynamic recurrent networks. *Neural Computation*, 4(1):131–139, 1992.
- [66] Noam Shazeer. Glue variants improve transformer, 2020.
- [67] Sam Shleifer, Jason Weston, and Myle Ott. Normformer: Improved transformer pretraining with extra normalization. *arXiv preprint arXiv:2110.09456*, 2021.

- [68] Assaf Shocher, Nadav Cohen, and Michal Irani. “zero-shot” super-resolution using deep internal learning. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 3118–3126, 2018.
- [69] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding, 2023.
- [70] Yu Sun, Xinhao Li, Karan Dalal, Chloe Hsu, Sanmi Koyejo, Carlos Guestrin, Xiaolong Wang, Tatsunori Hashimoto, and Xinlei Chen. Learning to (learn at test time). *arXiv preprint arXiv:2310.13807*, 2023.
- [71] Yu Sun, Wyatt L Ubellacker, Wen-Loong Ma, Xiang Zhang, Changhao Wang, Noel V Csomay-Shanklin, Masayoshi Tomizuka, Koushil Sreenath, and Aaron D Ames. Online learning of unknown dynamics for model-based controllers in legged locomotion. *IEEE Robotics and Automation Letters*, 6(4):8442–8449, 2021.
- [72] Yu Sun, Xiaolong Wang, Zhuang Liu, John Miller, Alexei Efros, and Moritz Hardt. Test-time training with self-supervision for generalization under distribution shifts. In *International Conference on Machine Learning*, pages 9229–9248. PMLR, 2020.
- [73] Sebastian Thrun and Lorien Pratt. Learning to learn: Introduction and overview. In *Learning to learn*, pages 3–17. Springer, 1998.
- [74] Tijmen Tieleman and Geoffrey Hinton. Using fast weights to improve persistent contrastive divergence. In *Proceedings of the 26th annual international conference on machine learning*, pages 1033–1040, 2009.
- [75] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. Llama 2: Open foundation and fine-tuned chat models, 2023.
- [76] Vladimir Vapnik. *The nature of statistical learning theory*. Springer science & business media, 2013.
- [77] Pascal Vincent, Hugo Larochelle, Yoshua Bengio, and Pierre-Antoine Manzagol. Extracting and composing robust features with denoising autoencoders. In *ICML*, page 1096–1103, 2008.
- [78] Christoph Von Der Malsburg. The correlation theory of brain function. In *Models of neural networks: Temporal aspects of coding and information processing in biological systems*, pages 95–119. Springer, 1994.
- [79] Renhao Wang, Yu Sun, Yossi Gandelsman, Xinlei Chen, Alexei A Efros, and Xiaolong Wang. Test-time training on video streams. *arXiv preprint arXiv:2307.05014*, 2023.
- [80] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, et al. Huggingface’s transformers: State-of-the-art natural language processing. *arXiv preprint arXiv:1910.03771*, 2019.

- [81] Wenhan Xiong, Jingyu Liu, Igor Molybog, Hejia Zhang, Prajjwal Bhargava, Rui Hou, Louis Martin, Rashi Rungta, Karthik Abinav Sankararaman, Barlas Oguz, Madian Khabisa, Han Fang, Yashar Mehdad, Sharan Narang, Kshitiz Malik, Angela Fan, Shruti Bhosale, Sergey Edunov, Mike Lewis, Sinong Wang, and Hao Ma. Effective long-context scaling of foundation models, 2023.
- [82] Songlin Yang, Bailin Wang, Yikang Shen, Rameswar Panda, and Yoon Kim. Gated linear attention transformers with hardware-efficient training. *arXiv preprint arXiv:2312.06635*, 2023.
- [83] Songlin Yang, Bailin Wang, Yu Zhang, Yikang Shen, and Yoon Kim. Parallelizing linear transformers with the delta rule over sequence length. *arXiv preprint arXiv:2406.06484*, 2024.
- [84] Biao Zhang and Rico Sennrich. Root mean square layer normalization, 2019.
- [85] Hao Zhang, Alexander C Berg, Michael Maire, and Jitendra Malik. Svm-knn: Discriminative nearest neighbor classification for visual category recognition. In *2006 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'06)*, volume 2, pages 2126–2136. IEEE, 2006.

对偶形式

本节推导具有非线性激活的任意深度通用多层感知机的对偶形式。

不失一般性，为方便起见考虑 $\eta = 1$ ，且仅考虑第一个小批量，其中 $t = 1, \dots, b$ 。记： $\hat{x}_t = \theta_K x_t$ ， $y_t = \theta_V x_t$ ， $\bar{x}_t = \theta_Q x_t$ 。

同样记 $\hat{X} = [\hat{x}_1, \dots, \hat{x}_b]$ ，并类似地记 Y 和 $\bar{X}$ 。通常，大写字母表示矩阵，其列向量由对应的小写字母表示。

对于具有 K 层的网络，将第 k 层的初始参数记为 W_0^k 。本文约定用上标表示层，下标表示时间。

A.1 前向传播

在 TTT 的初始前向传播过程中，将第 k 层的输入记为 $\hat{X}^k = [\hat{x}_1^k, \dots, \hat{x}_b^k]$ ，其中 $\hat{X}^1 = \hat{X}$ 。现在使用这些记号写出 TTT 的前向传播过程。

对 $k = 1, \dots, K$:

- $Z^k = W_0^k \hat{X}^k$
- $\hat{X}^{k+1} = \sigma_k(Z^k)$

其中 σ_k 对于 $k = 1, \dots, K$ 可以是任意逐元素运算 ($\mathbb{R} \rightarrow \mathbb{R}$)，其导数为 $\sigma'_k(\cdot)$ 。给定 $\hat{X}^{K+1}$ ，我们计算损失：

$$l = \frac{1}{2} \ell(W_0^1, \dots, W_0^K; X) = \frac{1}{2} \|\hat{X}^{K+1} - Y\|_F^2 = \sum_{t=1}^b l_t,$$

其中 $l_t = \frac{1}{2} \|\hat{x}_t^K - y_t\|^2$ 与方程 4 中的定义相同，只是为方便起见按 $1/2$ 进行了缩放。

上述所有运算（除 σ 外）均为矩阵乘法和求和，因此具有硬件高效性。原始形式和对偶形式共享这些初始运算。

A.2 原始形式

原始形式首先计算 $G_t^k = \nabla_{W_0^k} l_t$ 对 $t = 1, \dots, b$ $W_t^k = W_0^k - \sum_{s=1}^t G_s^k$. Finally,
given $\bar{X}^1 = [\bar{x}_1^1, \dots,$

对 $k = 1, \dots, K$: $\bar{x}_b^k = \bar{x}_b^k$ ，然后更新 $\bar{x}_b^k = \bar{x}_b^k$ ，原始形式使用更新后的 W s。重复前向传播

- $\bar{z}_t^k = W_t^k \bar{x}_t^k$, for $t = 1, \dots, T$
- $\bar{x}_t^{k+1} = \sigma_k(\bar{z}_t^k)$, for $t = 1, \dots, T$

其中 $\bar{X}^{K+1} = [\bar{x}_1^{K+1}, \dots, \bar{x}_b^{K+1}]$ 包含输出标记。

注意，标准反向传播仅计算梯度之和：

$$\nabla_{W_0^k} l = \sum_{t=1}^b \nabla_{W_0^k} l_t = \sum_{t=1}^b G_t^k,$$

因此，求和 G_t^k 中针对 $t = 1, \dots, b$ 的各个项无法批量合并为矩阵乘法。类似地，原始形式的前向传播对每个 $\bar{x}_t$ 使用不同的 W_t ，因此也无法像标准前向传播那样批量处理。这些非标准传播的硬件效率较低。

A.3 对偶形式

如第 2.5 小节所述，对偶形式的目标是仅通过矩阵乘法和轻量级操作（如求和 s , σ 及 σ' ）来计算 $\bar{X}^{K+1}$ 和 $W_b^1, \dots, W_b^K$ 。为实现这一目标，我们避免显式计算中间变量： G_t^k 和 W_t^k （针对 $t = 1, \dots, b$ ）。

对偶形式首先计算 $\nabla_{\hat{X}^{K+1}} l = \hat{X}^{K+1} - Y$ ，然后执行标准反向传播。

对 $k = K, \dots, 1$:

- $\nabla_{Z^k} l = \sigma'_k(Z^k) \odot \nabla_{\hat{X}^{k+1}} l$
- $\nabla_{\hat{X}^k} l = (W_0^k)^T \nabla_{Z^k} l$
- $\nabla_{W_0^k} l = \nabla_{Z^k} l (\hat{X}^k)^T$

其中 σ' 按元素应用， $\odot$ 为逐元素乘法。

现在我们已经可以计算 $W_b^k = W_0^k - \nabla_{W_0^k} l$ 。为了计算输出标记，我们进行另一次前向传播。

对 $k = 1, \dots, K$:

- $\bar{Z}^k = W_0^k \bar{X}^k - \nabla_{Z^k} l \cdot \text{mask}\left((\hat{X}^k)^T \bar{X}^k\right)$
- $\bar{X}^{k+1} = \sigma(\bar{Z}^k)$

在前向传播结束时，我们已经计算出了 $\bar{X}^{K+1}$ 。

虽然这种前向传播是非标准的，但它只包含矩阵乘法、求和 s , σ 和掩码操作，因此与标准前向传播一样高效。

A.4 推导

为了推导对偶形式，我们证明：

$$\bar{Z}^k = W_0^k \bar{X}^k - \nabla_{Z^k} l \cdot \text{mask}\left((\hat{X}^k)^T \bar{X}^k\right)$$

与原始形式中计算的结果相同。具体来说，我们证明对偶形式第二次前向传播中 $\bar{Z}^k$ 的每一列 $\bar{z}_t^k$ 等于原始形式前向传播中的 $W_t^k \bar{x}_t^k$ 。我们引用一个简单的事实。

Fact 1. Define matrices $A = [a_1, \dots, a_b]$, $Q = [q_1, \dots, q_b]$, and $V = [v_1, \dots, v_b]$.¹⁴ Define $\hat{v}_t = \sum_{s=1}^t a_s^T q_t v_s$, and $\hat{V} = [\hat{v}_1, \dots, \hat{v}_b]$, then $\hat{V} = V \cdot \text{mask}(A^T Q)$.

现在将 $A = \hat{X}^k$, $Q = \bar{X}^k$, $V = -\nabla_{Z^k} l$ 和 $\hat{V} = W^k \bar{X}^k - \bar{Z}^k$ 代入上述事实，我们已经证明了所需的等式。

注意，上述使用的 σ_k 和 σ'_k 可以扩展到任意函数，这些函数不一定是逐元素操作，包括归一化层。这种扩展可以通过例如 JAX 和 PyTorch 等自动微分标准库中的 vjp（向量-雅可比乘积）来实现。然而，对偶形式不能加速 σ 内部或其 vjp 中的操作。

¹⁴ 我们的矩阵 A 在另一种语境下通常记作 K 。我们使用 A 以避免与层数 K 混淆。

B Nadaraya-Watson 估计器

Nadaraya-Watson 估计器的推导。在本节中，我们用 $\mathbf{x}$ 表示输入标记 $\mathbf{x}$ 作为随机变量。我们期望的输出是对应的输出标记，即另一个随机变量 $\mathbf{z}$ 。这被形式化为估计 $\mathbf{z}$ 的条件期望：

$$\mathbb{E}[\mathbf{z}|\mathbf{x} = x] = \int p(\mathbf{z}|x) \mathbf{z} d\mathbf{z} = \int \frac{p(x, \mathbf{z})}{p(x)} \mathbf{z} d\mathbf{z}.$$

由于真实概率分布 $p(x)$ 和 $p(x, \mathbf{z})$ 未知，我们用它们的核密度估计来替代。具体来说， $p(x)$ 的核密度估计为：

$$\hat{p}(x) = \frac{1}{n} \sum_{i=1}^n \kappa(x, x_i),$$

其中每个 x_i 通常是一条训练数据。（回顾本文， x_i 特指内循环的训练数据，即一个标记，这与正文中的记号一致。）对于估计 $p(x, \mathbf{z})$ ，我们使用乘积核：

$$\hat{p}(x, \mathbf{z}) = \frac{1}{n} \sum_{i=1}^n \kappa(x, x_i) \kappa'(z, z_i).$$

乍一看，将联合概率分解为两个看似独立的核似乎很荒谬。但在这种情况下， κ' 实际上可以是任何依赖于 x_i 的 κ'_i ，因为它会被积分掉。因此这两个核不需要独立。

代入这些估计，我们得到 Nadaraya-Watson 估计器：

$$\begin{aligned} \hat{\mathbb{E}}[\mathbf{z}|\mathbf{x} = x] &= \int \frac{\hat{p}(x, \mathbf{z})}{\hat{p}(x)} \mathbf{z} d\mathbf{z} \\ &= \frac{1}{\hat{p}(x)} \int \hat{p}(x, \mathbf{z}) \mathbf{z} d\mathbf{z} \\ &= \frac{1}{\sum_{i=1}^n \kappa(x, x_i)} \int \sum_{i=1}^n \kappa(x, x_i) \kappa'(z, z_i) \mathbf{z} d\mathbf{z} \\ &= \frac{1}{\sum_{i=1}^n \kappa(x, x_i)} \sum_{i=1}^n \kappa(x, x_i) \int \kappa'(z, z_i) \mathbf{z} d\mathbf{z} \\ &= \frac{1}{\sum_{i=1}^n \kappa(x, x_i)} \sum_{i=1}^n \kappa(x, x_i) z_i. \end{aligned}$$

非对称核函数。在现代，人们将核函数视为半正定的，这对于 κ 可能无法保证，除非 $\theta_K = \theta_Q$ 。然而，几十年前研究核函数的人，大约在纳达拉亚-沃森估计量流行的时期，对核函数的选择非常宽松，像我们方程 10 中的 κ 这样的非对称核函数有着悠久的传统：当核估计器使用 $\theta_K \neq \theta_Q$ 时，它被称为气球估计器 [14]。像布雷曼等人 [10] 的论文甚至将 θ_Q 用作 x' 的函数，这被称为样本自适应平滑。

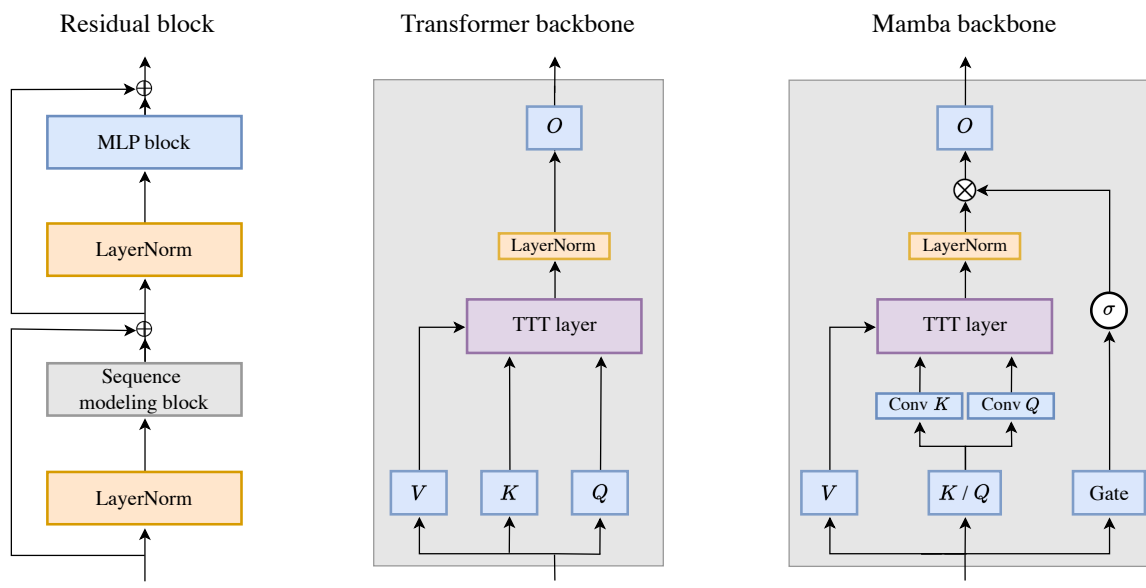


图 13. 左：残差块，变换器的基本构建模块。序列建模模块实例化为两种变体：变换器骨干和曼巴骨干。中：变换器骨干中的 TTT 层。O 之前的 LN 来自规范变换器 [67]。右：受曼巴 [27] 和格里芬 [18] 启发的骨干中的 TTT 层。遵循这两种架构，这里的 σ 是 GELU[31]。为了在不改变嵌入维度的情况下容纳门的额外参数，我们简单地将 θ_K 和 θ_Q 合并为一个投影。

C 实验细节

架构。我们的变换器严格遵循曼巴论文中的构造，其中变换器被称为 Transformer++。具体来说，变换器架构基于 Llama[75]，使用旋转位置编码（RoPE）[69]、SwiGLU MLP 块 [66] 和 RMSNorm[84] 代替层归一化。我们的曼巴基线使用作者提供的公开代码。我们已验证我们的基线能够重现 [27] 中报告的数字。

训练配置。我们的训练配置在表 3 中，该表简单地复现了

Mamba 论文中的表 12。如脚注 12 所述，所有模型均以 0.5M 个标记的批大小进行训练，与上下文长度无关。我们所有的优化超参数均遵循 Mamba 论文附录 E.2 中的“改进配方”，现转载如下：

- AdamW optimizer: $\beta = (0.9, 0.95)$
- 余弦调度：衰减至最终学习率 $1e-5$ • 在训练步数的 10% 内进行线性学习率预热 • 权重衰减：0.1 • 梯度裁剪：1.0 • 无随机失活 • 混合精度

Params.	Blocks	Embed. dim.	Heads	Train steps	Peak LR	Tokens
125M	12	768	12	4800	3e-3	2.5B
350M	24	1024	16	13500	1.5e-3	7B
760M	24	1536	16	29000	1.25e-3	15B
1.3B	24	2048	32	50000	1e-3	26B

表 3. 所有实验的训练配置。本表复现了 Mamba 论文中的表 12。唯一区别在于，他们为 Mamba 和 Transformer 使用的学习率是表 12 中数值的 $5\times$ ，而我们报告实际值 ($5\times$)。注意，本表仅适用于 TTT-Linear、TTT-MLP 和 Transformer，因为 Mamba 不遵循继承自 Transformer 的多头残差块结构。

如脚注 10 所述，所有模型均使用 Llama 分词器 [75] 进行训练。对于 Pile 上的实验，这是与 Mamba 论文配方的唯一区别，后者使用了另外两种分词器。对于 Books 上的实验，我们发现 RoPE 编码的原始角度 [69] $\theta = 10,000$ 对于我们的 Transformer 基线在长上下文下并非最优。从上下文长度 4k 开始，我们尝试 $\theta = 500,000$ （遵循 Llama Long 论文 [81]），并为 Transformer 使用更好的困惑度（预训练和微调均如此）。

变换器微调。微调开始一个新的余弦调度，除峰值学习率外，优化超参数与从头训练相同。我们尝试了三种微调峰值学习率：1e-5、1e-4 和 1e-3，并选择最佳困惑度。我们观察到，对于 125M 模型，1e-4 效果最佳；对于 350M 及更大模型，1e-5 效果最佳。考虑到 Chinchilla 配方的最终学习率为 1e-5，这一观察是合理的。

测试时训练 (TTT) 的学习率。如第 2.7 小节所述，内循环基础学习率 η_{base} 对于 TTT-Linear 设为 1，对于 TTT-MLP 设为 0.1。我们设置 η_{base} 的启发式方法类似于人们为常规训练设置外循环学习率的方式：我们尝试了 $\eta_{\text{base}} \in \{0.01, 0.1, 1, 10\}$ ，并使用不会导致不稳定性的最大值。对于 TTT-MLP，我们在训练步数的 10% 内对 η_{base} 使用线性预热，类似于常规训练。内循环中的训练步数为 T/b （假设可整除）。对于 TTT-Linear，我们尝试了内循环中的线性预热，但未观察到差异。

图实验 图 2（右）。为确保对曼巴 (Mamba) 的公平性，这些实验中的所有方法均匹配训练浮点运算次数 (FLOPs)，并采用与曼巴 1.4B 相同的训练方案（表 3 最后一行）。对于 TTT-Linear 和 TTT-MLP，匹配训练浮点运算次数也意味着匹配推理浮点运算次数。变换器 (TF 微调) 的推理浮点运算次数为 $.8\times$ 倍，这使其作为基线具有优势。为与曼巴匹配训练浮点运算次数，变换器使用 19 个块而非 **24** 个。对于 TTT-Linear 和 TTT-MLP，其训练浮点运算次数已接近曼巴，因此只需将 MLP 块的隐藏维度从 5504 调整为 TTT-Linear 的 5808 和 TTT-MLP 的 **5248**。

时间维度的梯度检查点。默认情况下，JAX 和 PyTorch 等库在前向传播过程中保存中间激活，以便在反向传播中重用。然而，对于以 W 为隐藏状态的 TTT 层，此默认设置会保存 $W_1, \dots, W_T$ ，占用过多内存。使用 TTT 小批量和双形式，我们仍需在大批量结束时保存（假设可整除） $\kappa = T/b$ 个 W 。在此场景下节省内存的标准技术是梯度检查点 [12]，通常按层应用，但我们按时间应用。

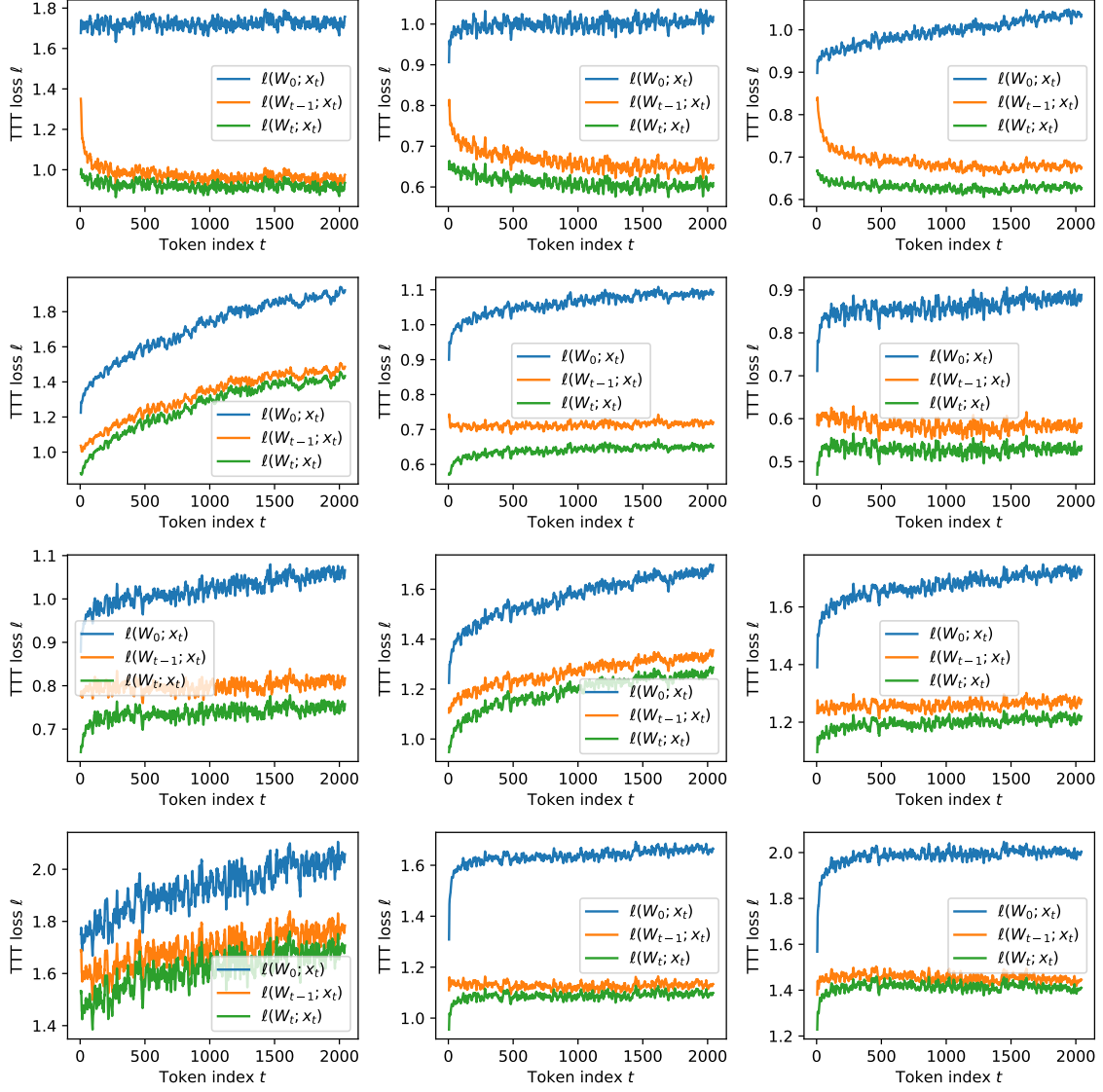


图 14。自监督 TTT 损失 ℓ 在所有测试序列上的平均值，序列形式为 $x_1, \dots, x_T$ ，其中 $T = 2048$ ，针对在 Pile 上训练的具有 125M 参数的网络中所有 12 个 TTT 层。同一网络也用于图 7 左面板中的 $b=1$ （在线梯度下降）。对于中间层，我们观察到 $\|x_t\|$ 稳步上升，导致三种损失随之上升。即使对于这些层， $\ell(W_0; x_t)$ 与 $\ell(W_t; x_t)$ 之间的差距仍随 t 增大。为清晰起见，损失值已在 10 个时间步的滑动窗口上取平均。

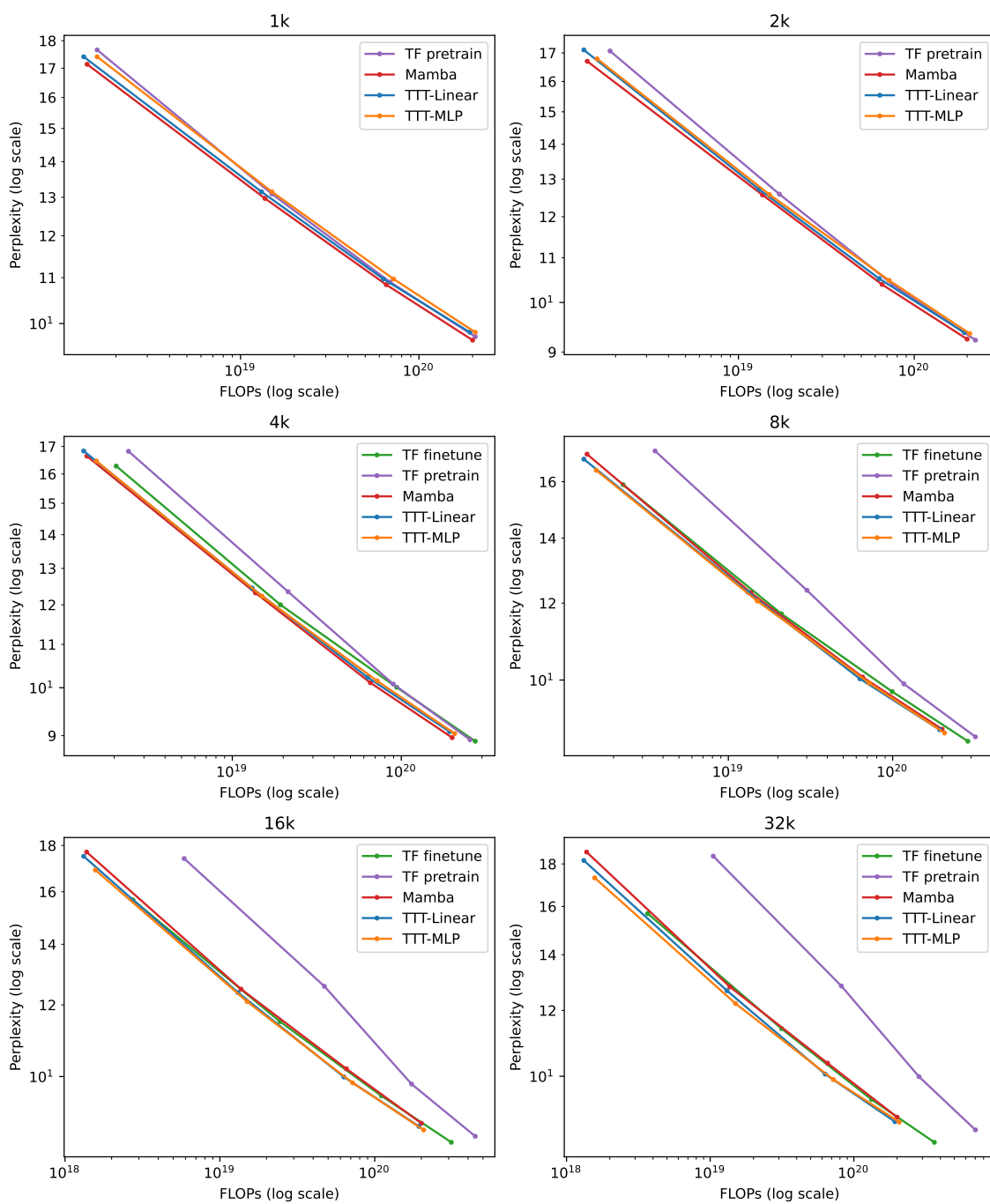


图 15。按上下文长度呈现的 Books 完整结果。第 3.2 小节中的图 11 展示了上下文长度为 2k 和 32k 的部分结果。

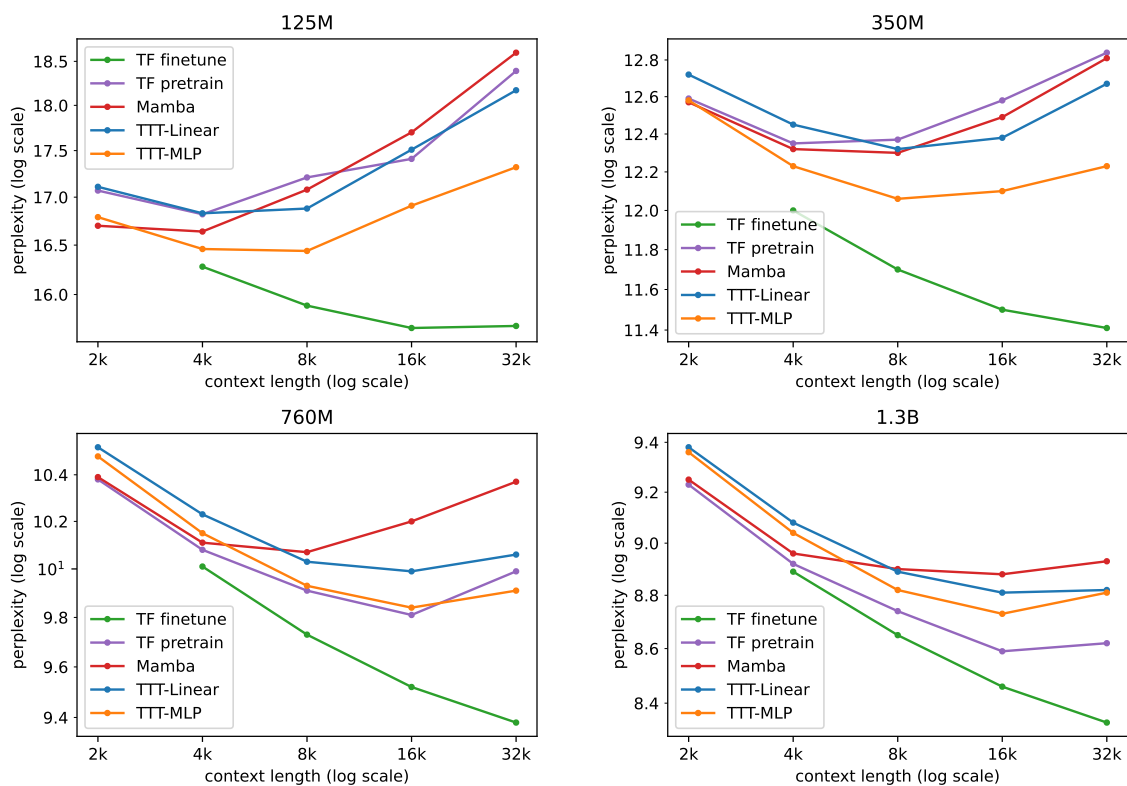


图 16。按模型规模呈现的 Books 完整结果的另一种视图，以上下文长度为 x 轴。对于所有从头训练的方法，一旦上下文长度过大，困惑度会变差。此趋势在 TF 微调中未观察到，除了 125M 规模的一个案例。最佳上下文长度随模型规模增大而增加（从头训练）。